Chapter 3

Problem-solving agents

- Reflex agent is simple
  - based their actions on
  - a direct mapping from states to actions
  - but cannot work well in environments
    - which this mapping would be too large to store
    - and would take too long to learn
- Hence, goal-based agent is used

Problem-solving agent

- Problem-solving agent
  - A kind of goal-based agent
  - It solves problem by
    - finding sequences of actions that lead to desirable states (goals)
  - To solve a problem,
    - the first step is the ***goal formulation***, based on the current situation
    - Goals help organize behavior by limiting the objectives that the agent is trying to achieve and hence the actions it needs to consider.

Steps to maximize the performance measure

- Goal formulation
- Problem formulation
- Search
- Execution

Goal formulation

- The goal is formulated
  - as a set of world states, in which the goal is satisfied or Decide what the agent wants to achieve.
- Reaching from initial state → goal state
  - Actions are required
- *Actions* are the operators
  - causing transitions between world states
  - **Actions** should be abstract enough at a certain degree, instead of very detailed
  - E.g., **turn left** VS **turn left 30 degree**, etc.

Problem formulation

- The process of defining,
  - what actions and states to consider given a goal.
- E.g., driving Amman → Zarqa
  - in-between states and actions defined
  - States: Some places in Amman & Zarqa
  - Actions: Turn left, Turn right, go straight, accelerate & brake, etc.

Search

- Because there are many ways to achieve the same goal
  - Those ways are together expressed as a tree
  - Multiple options of unknown value at a point,
    - the agent can examine different possible sequences of actions, and choose the best
  - Explore different possible sequences of actions to find a path from start state to goal state is called ***search***
  - The best sequence is then a list of actions, called ***solution***

Execution

- Once the solution has found, the actions that are recommended by serach process can be carried out and it is known as execution phase.

Search algorithm

- Defined as
  - taking a *problem*
  - and returns a *solution*
- Once a solution is found
  - the agent follows the solution
  - and carries out the list of actions – execution phase
- Design of an problem solving agent involves
  - “Formulate, search, execute”

A simple problem-solving agent program

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, some description of the current world state
              goal, a goal, initially null
              problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
      goal ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq ← SEARCH(problem)
      if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

// seq = series of Actions

// first (seq) will select the first action in a sequence of actions and rest(seq) will return remove first action and returns the remaining actions as sequence.

Figure 3.1 A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

Well-defined problems and solutions

- A well-defined problem is one where all the components of problem formulation are clearly specified, so that a problem-solving agent can search for a solution systematically.

A well defined problem is defined by 5 components:

1. ***Initial state***
2. ***Actions***
3. ***Transition model or (Successor functions)***
4. ***Goal Test.***
5. ***Path Cost.***

- A problem is defined by 4 components:

1. The **initial state**
   - that the agent starts in
2. **Possible Actions** – what the agent can do.
3. **Transition model:** description of what each action does or outcome of action.

-successor functions: refer to any state reachable from given state by a single action

-Initial state, actions and Transition model define the **state space**

- the set of all states reachable from the initial state by any sequence of actions.

A **path** in the state space:

- any sequence of states connected by a sequence of actions.

Contd.,

4. The ***goal test***

- Applied to the current state to test
  - if the agent is in its goal

-Sometimes there is an explicit set of possible goal states. (example: in Amman).

-Sometimes the goal is described by the properties

- instead of stating explicitly the set of states

- Example: Chess
  - the agent wins if it can capture the KING of the opponent on next move (checkmate).
  - no matter what the opponent does

Contd.,

5. A ***path cost*** function,
   - assigns a numeric cost to each path
   - = performance measure
   - denoted by ***g***
   - to distinguish the best path from others
- Usually the path cost is
   - the sum of the **step costs** of the individual actions (in the action list)

Well-defined problems and solutions

- The ***feasible solution*** of a problem is then
  - *a path from the initial state to a state satisfying the goal test*
- ***Optimal*** solution
  - the solution with lowest path cost among all solutions

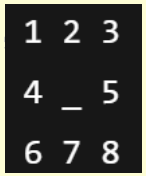
Ex.Travel from Arad to Destination

- Initial state: In(Arad)
- ACTIONS(s)

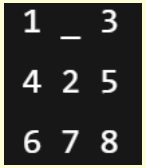
  { Go(Sibiu), Go(Timisoara), Go(Zerind) }
- The Transition model, specified by a function RESULT(s, a) that returns the state that results from TRANSITION MODEL doing action a in state s.
  RESULT(In(Arad),Go(Zerind)) = In(Zerind)
- The step cost of taking action a in state s to reach state s is denoted by c(s, a, s)

Ex.8-Puzzle Problem

- **Initial state**: Random arrangement of tiles.
- **Actions:** Slide one tile into the empty space. {MoveBlankLeft, MoveBlankRight, MoveBlankUp, MoveBlankDown}
- **Transition state**: Suppose the current state is

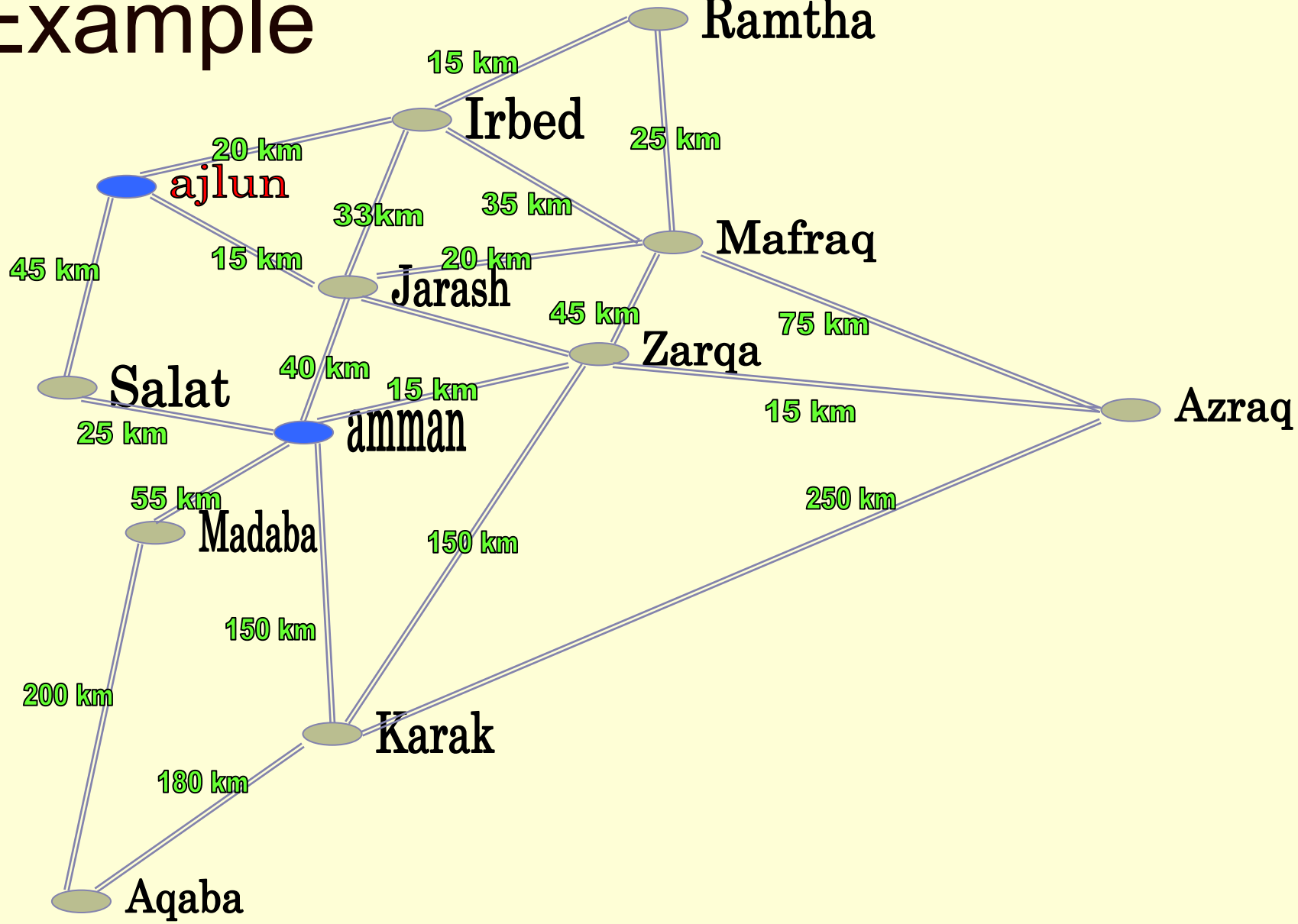


MoveBlankUp action will result in :



- **Goal test:** Tiles in correct order.
- **Path cost:** Number of moves.

Example


Ramtha
15 km
Irbed
20 km
ajlun
25 km
33km
35 km
Mafraq
45 km
15 km
20 km
Jarash
45 km
75 km
Zarqa
Salat
40 km
15 km
Azraq
25 km
amman
15 km
55 km
250 km
Madaba
150 km
150 km
200 km
Karak
180 km
Aqaba

From our Example

1. Formulate Goal

- Be In Amman

2. Formulate Problem

- States : Cities
- actions : Drive Between Cities

3. Find Solution

- Sequence of Cities : ajlun – Jarash - Amman

Our Example

1. **Problem : To Go from Ajlun to Amman**

2. **Initial State : In(Ajlween)**

3. **Operator : Go from One City To another .**

4. **State Space : {Jarash , Salat , irbed,……..}**

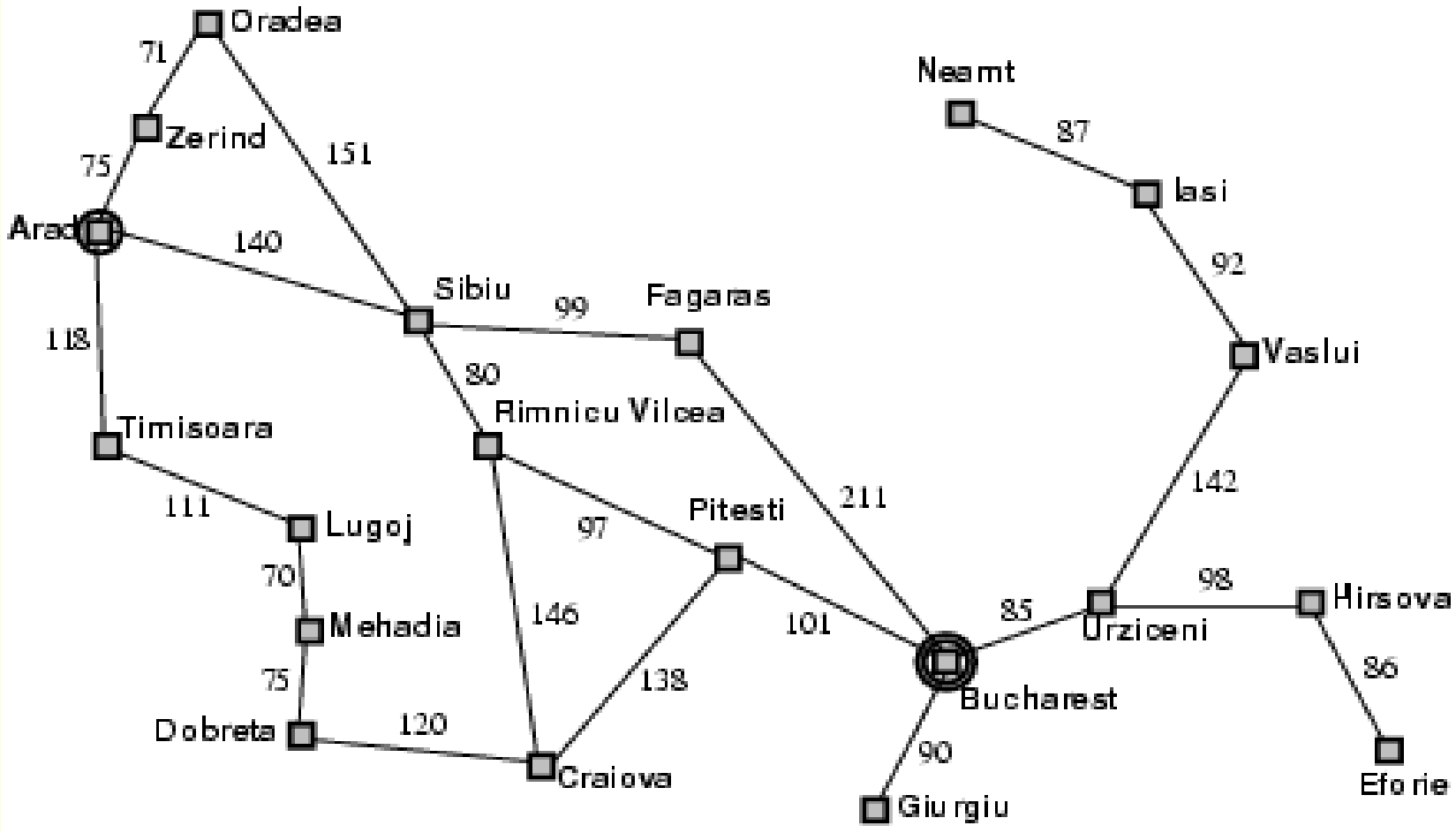
5. **Goal Test : are the agent in Amman.**

6. **Path Cost Function : Get The Cost From The Map.**

7. **Solution :** { {Aj → Ja → Ir → Ma → Za → Am} , {Aj →Ir → Ma → Za → Am} …. {Aj → Ja → Am} }

8. **State Set Space :** {Ajlun → Jarash → Amman}

Example: Romania



Single-state problem formulation

A problem is defined by four items:

1. initial state e.g., "at Arad"
2. actions or successor function *S(x)* = set of action–state pairs
   1. e.g., *S(Arad) = RESULT(IN(ARAB),GO(ZERIND)]=IN(Zerind))*
3. goal test, can be
   1. explicit, e.g., *x* = "at Bucharest"
   2. implicit, e.g., *Checkmate(x)*
4. path cost (additive)
   1. e.g., sum of distances, number of actions executed, etc.
   2. *c(x,a,y)* is the step cost, assumed to be ≥ 0

- A solution is a sequence of actions leading from the initial state to a goal state

Example problems

- *Toy problems*
  - those intended to illustrate or exercise various problem-solving methods
  - E.g., puzzle, chess, etc.
- *Real-world problems*
  - tend to be more difficult and whose solutions people actually care about
  - E.g., Design, planning, etc.

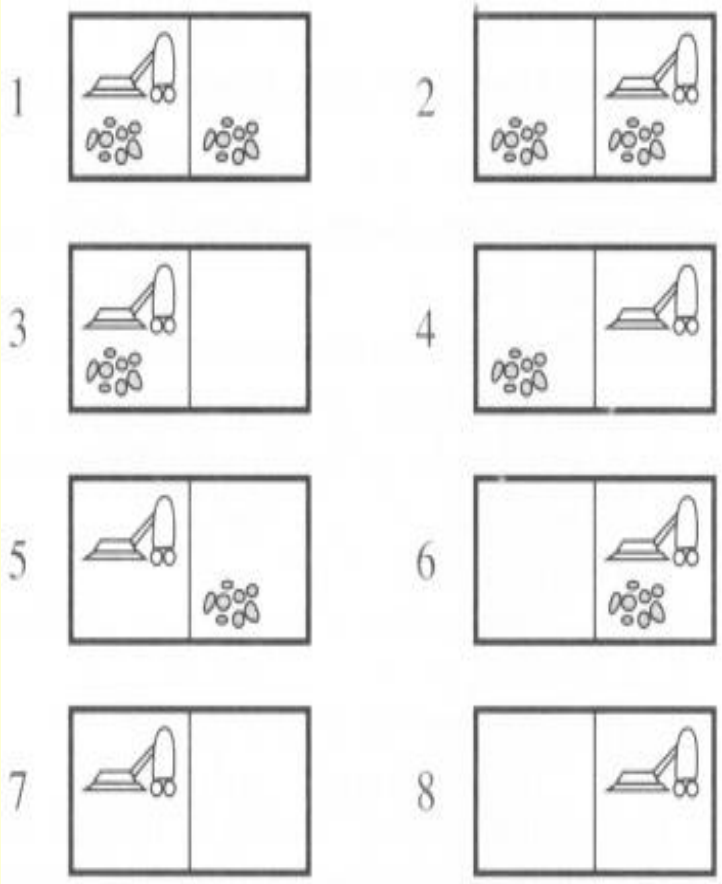
Toy problems

Example: vacuum world

- States: The state is determined by both the agent location and the dirt locations. The agent is in one of two locations, each of which might or might not contain dirt. Thus, there are $2 \times 2^2 = 8$ possible world states. A larger environment with n locations has $n \cdot 2^n$ states.

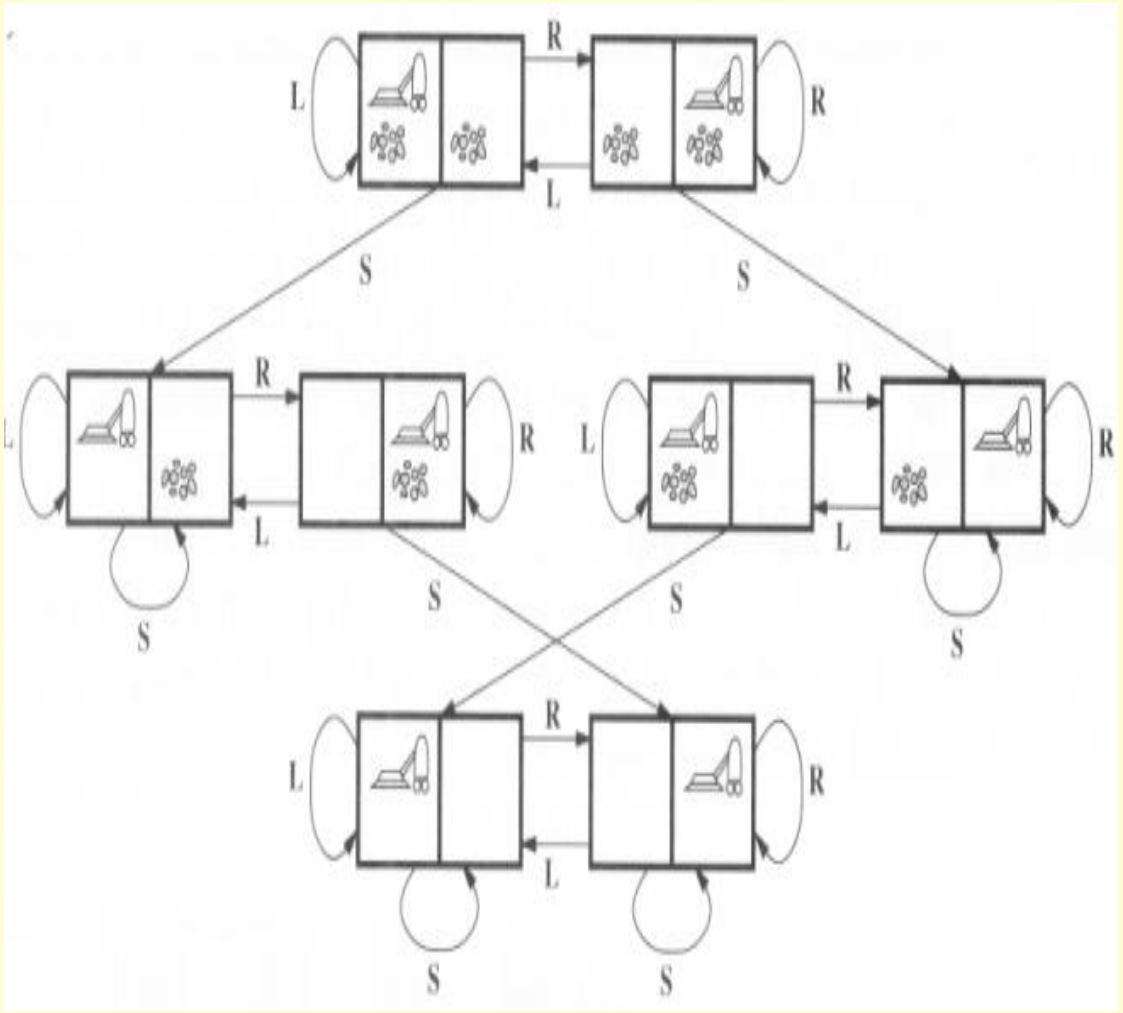
- Initial state: Any state can be designated as the initial state.

- Actions: In this simple environment, each state has just three actions: Left, Right, and Suck. Larger environments might also include Up and Down.



Toy problems

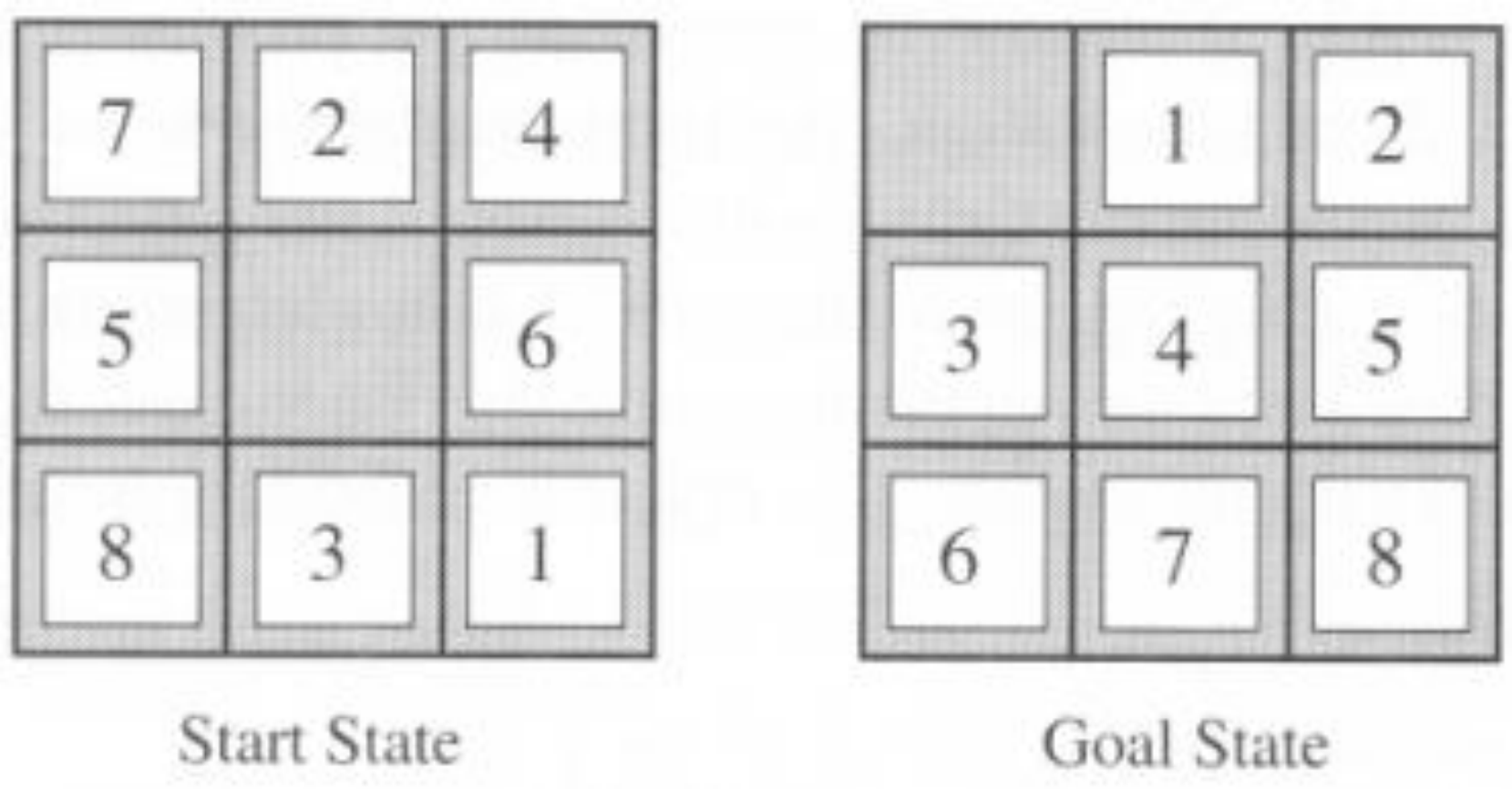
- **Transition model:** The actions have their expected effects, except that moving Left in the leftmost square, moving Right in the rightmost square, and Sucking in a clean square have no effect.



- Goal test: This checks whether all the squares are clean.

- Path cost: Each step costs 1, so the path cost is the number of steps in the path.

The 8-puzzle

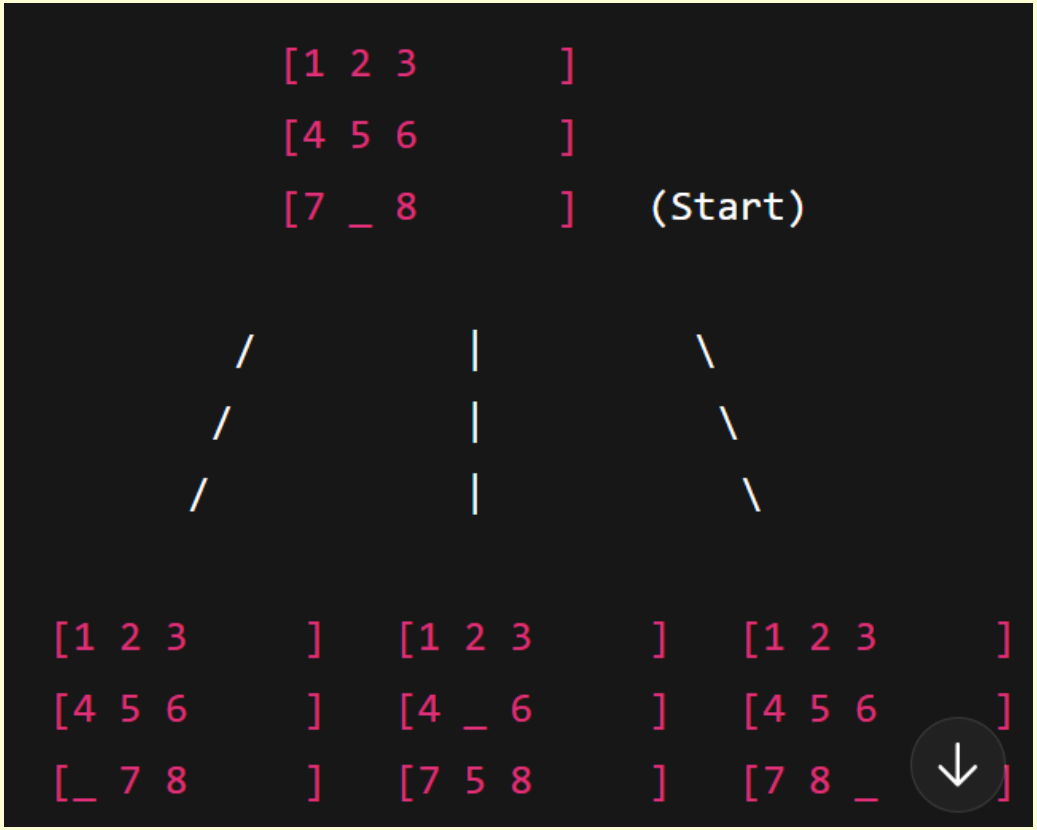


The 8-puzzle

- **States**:
  - a state description specifies the location of each of the eight tiles and blank in one of the nine squares
- **Initial State**:
  - Any state in state space
- **Actions:** movements of the blank space Left, Right, Up, or Down
- **Successor function**:
  - the blank tile movement results in switching between numbered tile and blank tile.*Result(s, a) → returns new state after action a*
- **Goal test**:
  - current state matches the goal configuration
- **Path cost**:
  - each step costs 1, so the path cost is just the length of the path

The 8-puzzle

- **Successor function**: (1 step of transition shown below)
  - the blank tile movement results in switching between numbered tile and blank tile.*Result(s, a) → returns new state after action a*



The 8-queens

- There are two ways to formulate the problem
- All of them have the common followings:
  - Goal test: 8 queens on board, not attacking to each other
  - Path cost: zero

The 8-queens

1) Incremental formulation

- States: Any arrangement of 0 to 8 queens on the board is a state.
- Initial state: No queens on the board.
- Actions: Add a queen to any empty square.
- Transition model: Returns the board with a queen added to the specified square.
- Goal test: 8 queens are on the board, none attacked

The 8-queens

2) Complete-state formulation

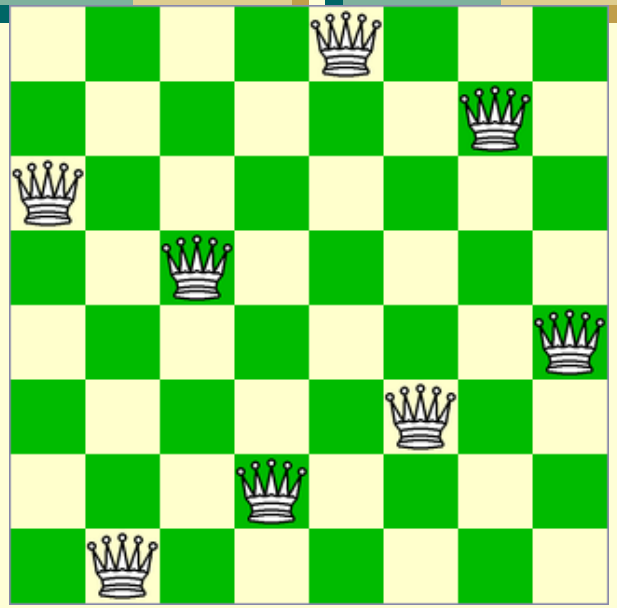
- States: All possible arrangements of n queens ($0 \leq n \leq 8$)on board, one per column in the leftmost n columns, with no queen attacking another.

- Actions: Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.

N-Queen Problem solving agent

- States → A configuration of queens on the board (partial or complete).
- Initial State → An empty chessboard.
- Actions → Place a queen in the next row in any safe column.
- Goal Test → N queens are placed without attacking each other.
- Path Cost → Not important here (all solutions have the same cost).

The 8-queens



- Solution Approaches
  1. **Backtracking Search**
     - Place queens row by row.
     - If conflict arises, backtrack and try another column.
     - Time-efficient for small n.
  2. **CSP** (constraint satisfaction problem)
  3. **Local search**

REAL WORLD PROBLEMS

- Route-finding problems
- Touring problems
- Traveling Salesman problem
- VLSI layout problem
- Robot navigation
- Automatic assembly sequencing
- Internet searching

1.Airline travel problems

- Typically refers to the complex optimization and planning problem faced by airlines, passengers, or systems related to airline operations, including flight scheduling, routing, pricing, or booking.

- **Passenger Travel Problem**: The Passenger Airline Travel Problem is a graph-based pathfinding + optimization problem where the passenger must find the best route (time/cost) from origin to destination given flight schedules, seat availability, and constraints.

Passenger Airline travel problem (Flight Booking)

- **Initial State:** The starting airport and time, with no flights taken.
- **Goal State:** Arrival at the destination airport within budget, time, and comfort constraints.
- **Intermediate States:** Represent locations (airports currently the passengers in) ,time and flights already taken while interchanging the flights.
- **Actions/Operators:** Selecting and booking available flights.
- **Constraints:** Time, cost, availability, and layover constraints.
- **Path Cost:** Combination of monetary cost (ticket cost), travel time, and comfort factors(like number of layovers, flight class (economy, business), and time of day (early morning or late night flights)).
- **Problem-Solving Strategy:** Graph search, dynamic programming, or heuristic search.
- **Feedback Mechanism:** Continuous evaluation of time, cost, and constraints at each step.

2. Touring Problem

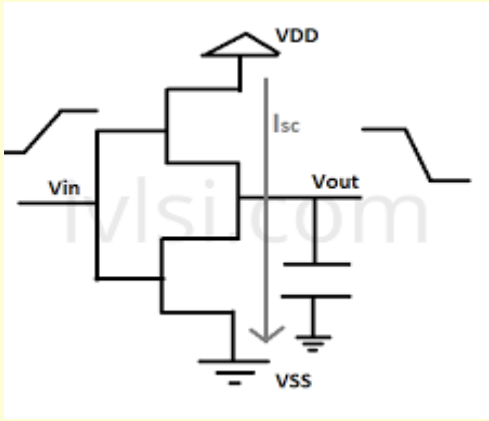
- The goal is to find the shortest possible route that visits each city or location exactly once and returns to the starting city.
- Typically referred to as **Travelling salesman problem**

TSP problem

- **Initial State:** The starting city from which the tour begins.
- **Goal State:** A complete tour visiting all cities exactly once and returning to the starting city.
- **Intermediate States:** Partial tours where some cities have been visited.
- **Actions/Operators:** Moving from the current city to an unvisited city.
- **Constraints:** Each city must be visited exactly once, and the tour must start and end at the same city.
- **Path Cost:** The total distance or travel cost of the tour, calculated from the distance/cost matrix.
- **Problem-Solving Strategy:** Includes exact methods (dynamic programming, branch and bound) and heuristics (nearest neighbor, simulated annealing, genetic algorithms, ant colony optimization).

3. VLSI LAYOUT problem

- It refers to the challenge of efficiently placing and routing electronic components (e.g., transistors, gates, wires) on a silicon chip to optimize performance, minimize area, reduce power consumption, and ensure proper functioning of the circuit.



VLSI LAYOUT PROBLEM

- **Initial State:** Abstract definition of the circuit (netlist / Cell layout) with unplaced and unrouted components.
- **Goal State:** A fully placed and routed layout that satisfies area, timing, power, and manufacturability constraints.
- **Intermediate States:** Partial placement or routing solutions with some components or connections still unplaced/unrouted.
- **Actions/Operators:** Placement of components, routing of wires, swapping/moving components, and adjusting paths.
- **Constraints:** Area, timing, routing, design rules, power, and congestion constraints.
- **Path Cost:** Combination of area, wirelength, timing, congestion, and power costs.
- **Problem-Solving Strategy:** Algorithms like simulated annealing, genetic algorithms, min-cut placement, and maze routing.
- **Feedback Mechanism**: Evaluation of cost and constraints after each move or routing step.

4. ROBOT Navigation Problem

- Is a classic problem in robotics and artificial intelligence that involves guiding a robot through an environment from an initial position to a goal position, avoiding obstacles and minimizing path costs (e.g., distance, time, energy)

ROBOT navigation problem

- **Initial State:** The robot's starting position, (x_start, y_start), and possibly its orientation or velocity.
- **Goal State:** The target position (x_goal, y_goal) or (x_goal, y_goal, θ_goal).
- **Intermediate States:** The robot's positions(x, y cordinates) as it navigates through the environment.
- **Actions/Operators:** The robot's available moves, such as moving up, down, left, right, or moving in continuous space with steering.
- **Constraints:** Includes avoiding obstacles(In a grid-based environment, obstacles are represented as blocked cells), respecting kinematic limits(if wheel robo - the limited turning radius), and staying within boundaries.
- **Path Cost:** A function that combines distance, time, energy, smoothness, and other factors.
- **Problem-Solving Strategies:** Uses algorithms such as BFS, A*, and dynamic programming to find optimal or near-optimal paths.

Automatic Assembly Sequencing problem

- The goal is to determine the optimal sequence of assembly operations for assembling a product.

Contd.,

- **Initial State:** The unassembled product, with all parts available but no connections made.
- **Goal State:** The fully assembled product, with all parts correctly connected.
- **Intermediate States:** Partial assemblies where some components are connected and some are not.
- **Actions/Operators:** Assembly operations that join parts or subassemblies.
- **Constraints:** Precedence, accessibility, tool, stability, geometric, and time constraints.
- **Path Cost:** The total cost of the assembly sequence, considering time, steps, tool changes, and other factors.
- **Problem-Solving Strategies:** Techniques include heuristic search, graph-based approaches, genetic algorithms, and constraint satisfaction.

3.3 Searching for solutions

3.3 Searching for solutions

- Finding out a solution is done by
  - searching through the state space systematically to find a sequence of actions (a path) from the start state to the goal state.
- All problems are transformed
  - as a search **tree**
  - Treats the state space as a **graph** (nodes = states, edges = actions).

Graph Search

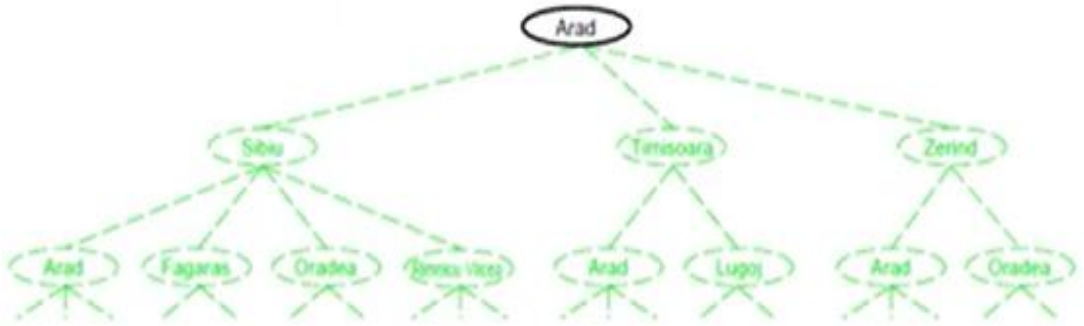
- In graph search, the search space is treated as a graph.
- Uses a visited/closed list (or explored set) to record already-explored states.
- Prevents re-exploring the same state or going in cycles.
- More memory-intensive (since it needs to keep track of explored states).

Tree Search

- In tree search, the search space is explored as a tree.
- Each node expansion generates new child nodes without checking if they have appeared before.
- Assumes no cycles or no repeated states.
- Simpler but can be inefficient (may re-explore the same state many times).

Searching for Solutions

- Together, the initial state and successor function implicitly define the **state space** of the problem.
- This lecture deals with **search techniques** that use an explicit **search tree** that is generated by the initial state and the successor function that together define the state space.
- E.g. Expansions in the *search tree for finding a route from Arad to Bucharest.*
- The root of the search tree is a **search node** corresponding to the initial state, *In(Arad).*
- The first step is to **test whether this is a goal state**. Clearly it is not

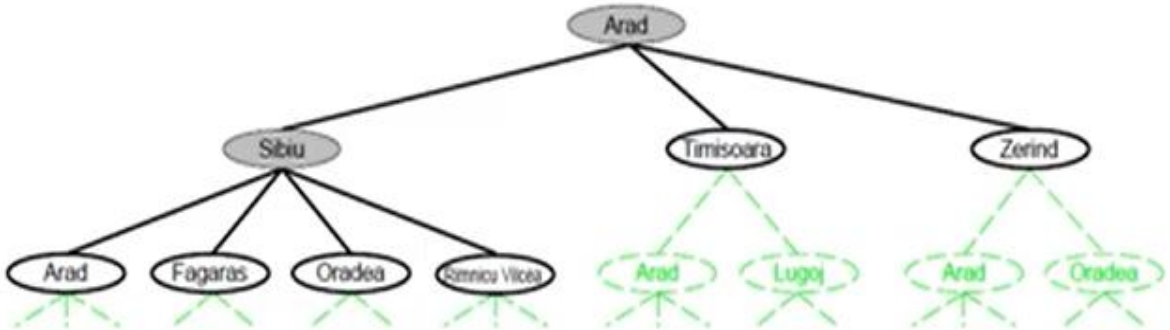


Searching for Solutions

- If the search node is not a goal state, we need to consider some other states.
- This is done by **expanding** the current state; that is, applying the successor function to the current state, thereby **generating** a new set of states.
- E.g In this case, we get three new states: *In(Sibiu), In(Timisoara),* and *In(Zerind).*
- Now we must choose which of these three possibilities consider further. **This is the essence of search.**
- ***following up one option now and putting the others aside for later, in case the first choice does not lead to a solution.***

Searching for Solutions

- E.g. Suppose we choose Sibiu first. We check to see whether it is a goal state (it is not) and then expand it to get ***In(Arad), In(Fagaras), In(Oradea), and In(RimnicuVi1cea).***
- We can then choose any of these four, or go back and choose Timisoara or Zerind.



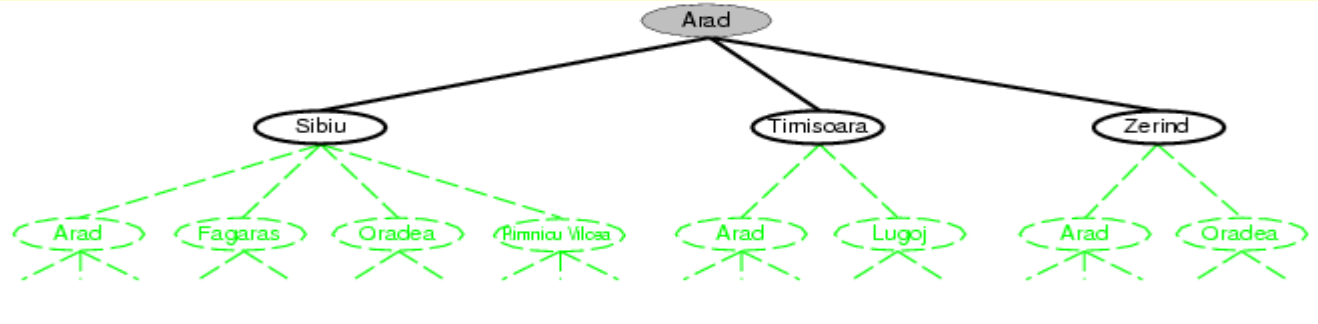
- We continue choosing, testing, and expanding until either a solution is found or there are no more states to be expanded. The choice of which state to expand is determined by the **search strategy.**

Search tree

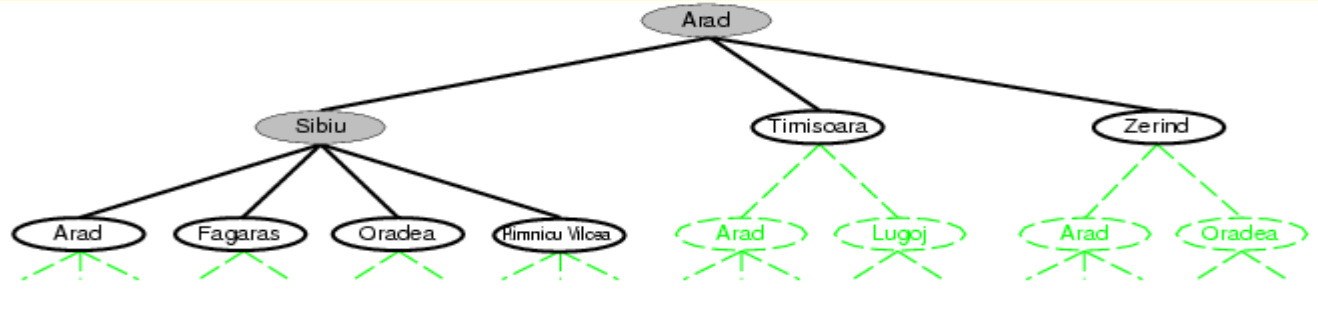
- **Initial state**
  - The root of the search tree is a **search node**
- **Expanding**
  - applying successor function to the current state
  - thereby generating a new set of states
- **leaf nodes**
  - the states having no successors

Fringe: Set of search nodes that have not been expanded yet.

Tree search example



Tree search example



Search tree

- The **essence** of searching
  - in case the first choice is not correct
  - choosing one option and keep others for later inspection
- Hence we have the **search strategy**
  - which determines the choice of which state to expand
  - good choice → fewer work → faster
- Important:
  - state space ≠ search tree

Contd.,

- The way to avoid exploring redundant/cyclic paths is to remember where one has been.
- To do this, we need to augment the TREE-SEARCH algorithm with a data structure called the explored set (also **EXPLORED SET** known as the **closed list**), which remembers every expanded node.
- Newly generated nodes that match previously generated nodes—ones in the explored set or the frontier—can be discarded instead of being added to the frontier.
- **frontier**: The set of all leaf nodes available for expansion at any given point is called the frontier.

Graph Search algorithm

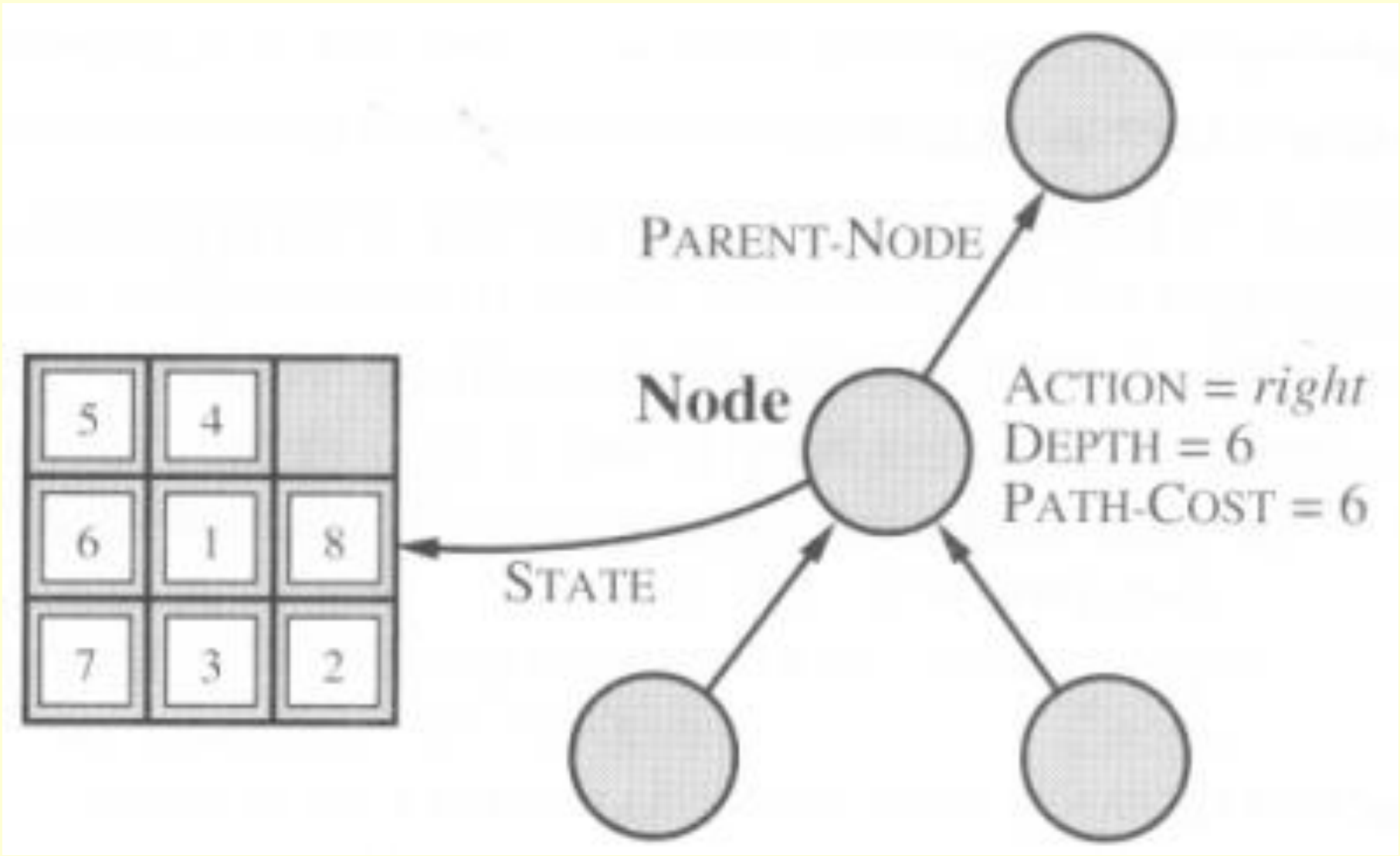
- **Key Components**

  - Frontier (Open list) → States generated but not yet explored (e.g., queue, stack, priority queue).
  - Explored set (Closed list) → States already visited.
  - Strategy → Defines how nodes are removed from the frontier:
  - FIFO → Breadth-First Search (BFS)
  - LIFO → Depth-First Search (DFS)
  - Priority (cost) → Uniform Cost Search
  - Priority (cost + heuristic) → A*

Graph Search algorithm

---

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  initialize the explored set to be empty
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set


PARENT-NODE
Node
ACTION = right
DEPTH = 6
PATH-COST = 6
STATE
5
4
6
1
8
7
3
2

Measuring problem-solving performance

- Properties of Search Algorithms
  - **Completeness:**
    - is the strategy guaranteed to find a solution when there is one?
  - **Optimality**:
    - does the strategy find the highest-quality solution when there are several different solutions?
  - **Time complexity**:
    - how long does it take to find a solution?
  - **Space complexity**:
    - how much memory is needed to perform the search?

Measuring problem-solving performance

- In AI, complexity is expressed in
  - **b**, branching factor, maximum number of successors of any node
  - **d**, the depth of the shallowest goal node.

  (depth of the least-cost solution)
  - **m**, the maximum length of any path in the state space
- Time and Space is measured in
  - number of nodes generated during the search
  - maximum number of nodes stored in memory

Measuring problem-solving performance

- For effectiveness of a search algorithm
  - we can just consider the total cost
  - **The total cost** = path cost (g) of the solution found **+** search cost
    - search cost = time necessary to find the solution
- Tradeoff:
  - (long time, optimal solution with least g)
  - vs. (shorter time, solution with slightly larger path cost g)

Types of Search Strategies

- Uninformed search
- Informed Search

3.4 Uninformed search strategies

https://youtu.be/20gd9bPlS_M?si=utNHue2oQOUCCXqV

3.4 Uninformed search strategies

- **Uninformed search**
  - no information about the number of steps

  or the path cost from the current state to the goal state other than what is provided in the problem definition.
  - search the state space **blindly** without considering how close it is to the goal.

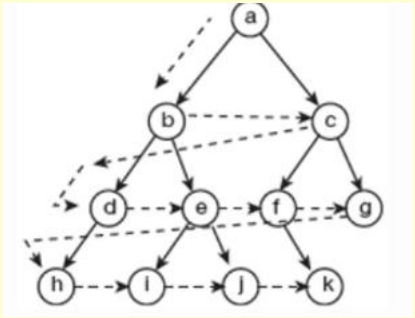
- **Informed search, or heuristic search** on other hand,
  - a cleverer strategy that searches toward the goal, based on the information from the current state so far.

Uninformed search strategies

- Breadth-first search
- Uniform cost search
- Depth-first search
  - Depth-limited search
  - Iterative deepening search
- Bidirectional search

1.Breadth-first search

- Breadth First Search (BFS) is an algorithm that traverses the tree breadthwise, meaning it explores all nodes at the current level before moving to the next level.



How It Works:

1.Start at the root node: The search begins at the root node (nodeA).

2.Traverse level by level: First, visit the immediate children of the root (nodes B and C).

3.Move to the next level: After visiting all nodes at the current level, the search moves to the next level, visiting nodes D, E, F, and G.

4.Continue level-wise traversal: The algorithm continues this process, exploring all neighbor nodes (children) at each level before moving deeper.

BFS

- **Key Components**

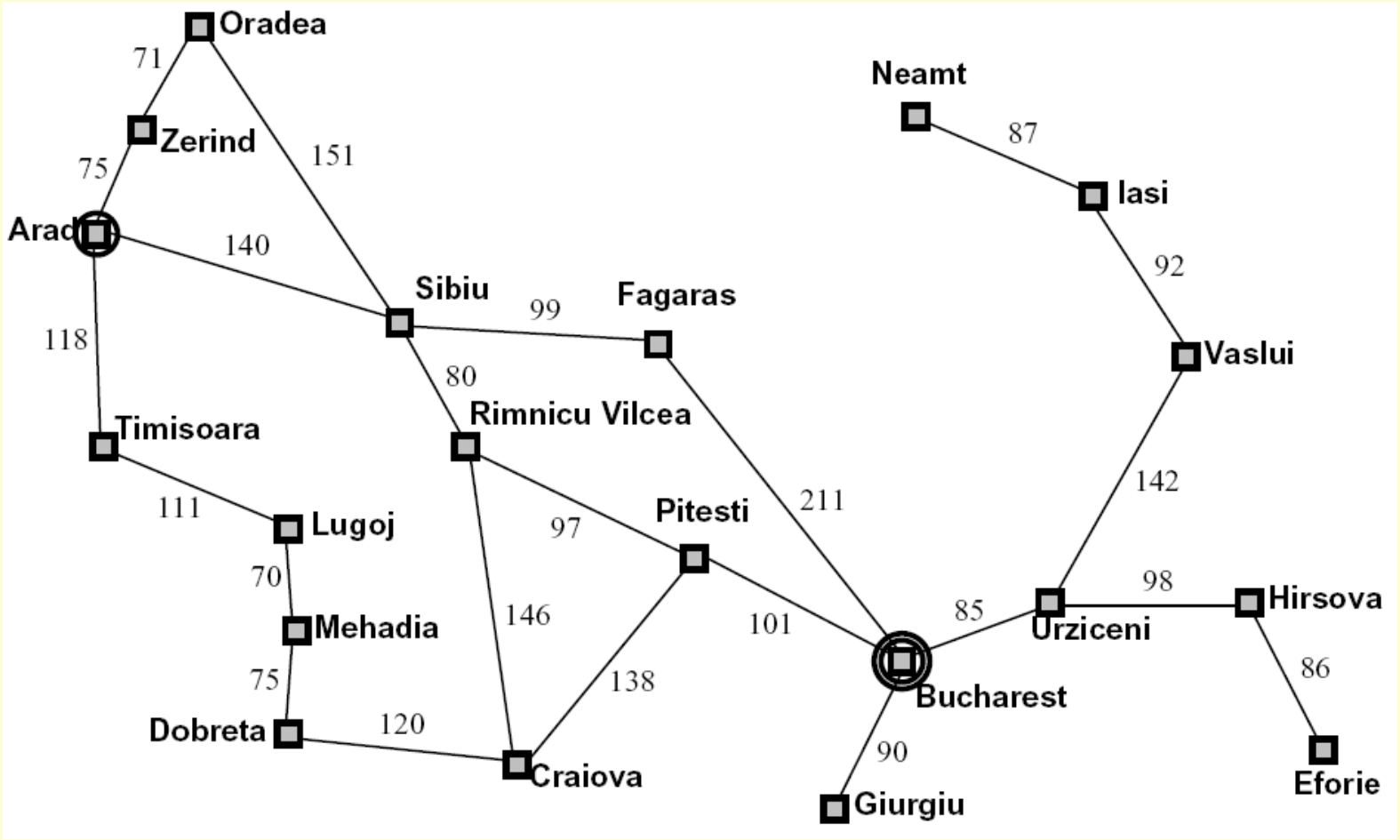
- Frontier (Queue)
  - Stores the nodes that are discovered but not yet expanded. Operates in FIFO order → First-In-First-Out.
- Explored List (Visited Set)
  - Stores nodes that have already been expanded, so they are not visited again.

BFS ALGORITHM

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
      if EMPTY?(frontier) then return failure
      node ← POP(frontier)  /* chooses the shallowest node in frontier */
      add node.STATE to explored
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          if child.STATE is not in explored or frontier then
              if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
              frontier ← INSERT(child, frontier)
```

Figure 3.11 Breadth-first search on a graph.

Example: Traveling from Arad to Bucharest



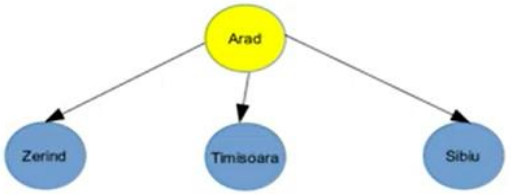


FIFO Queue

Frontier = [Arad]

Explored Set = []

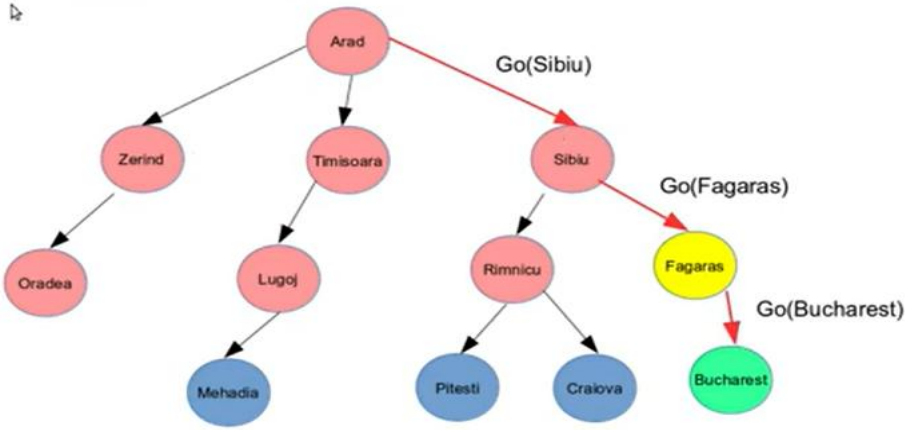
Valid Actions
Go(Zerind)
Go(Timisoara)
Go(Sibiu)



Frontier = [Zerind, Timisoara, Sibiu]

Explored Set = [Arad]

Solution = [Go(Sibiu), Go(Fagaras), Go(Bucharest)]



Frontier = [Mehadia, Pitesti, Craiova]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Lugoj, Rimnicu, Fagaras]

Breadth-first search (Analysis)

- Breadth-first search is,
  - Complete – find the solution eventually
  - Optimal, if step cost is 1

  The disadvantage
  - if the branching factor of a node is large,
    - the ***space complexity*** and the ***time complexity*** are enormous

Properties of breadth-first search

- Complete? Yes (if *b* is finite)
- Time? $1+b+b^2+b^3+\ldots+b^d = O(b^d)$ (Worst Case)

When goal test is applied when selected for expansion than when generated then it is - $O(b^{d+1})$

- Space? $O(b^{d+1})$ (keeps every node in memory)
- Optimal? Yes (if cost = 1 per step)
- Space is the bigger problem (more than time)

Breadth-first search (Analysis)

- assuming 10000 nodes can be processed per second, each with 1000 bytes of storage

Depth	Nodes	Time	Memory
2	1100	.11 seconds	1 megabyte
4	111,100	11 seconds	106 megabytes
6	10^7	19 minutes	10 gigabytes
8	10^9	31 hours	1 terabytes
10	10^{11}	129 days	101 terabytes
12	10^{13}	35 years	10 petabytes
14	10^{15}	3,523 years	1 exabyte

Figure 3.11 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 10,000 nodes/second; 1000 bytes/node.

2. Depth-first search

- Always expands one of the nodes at the ***deepest*** level of the tree
- Uses LIFO as frontier list
- Only when the search hits a dead end
  - backtrack and expands nodes at shallower levels
  - Dead end → leaf nodes but not the goal
- Backtracking search
  - only one successor is generated on expansion
  - rather than all successors
  - fewer memory

Depth-first search

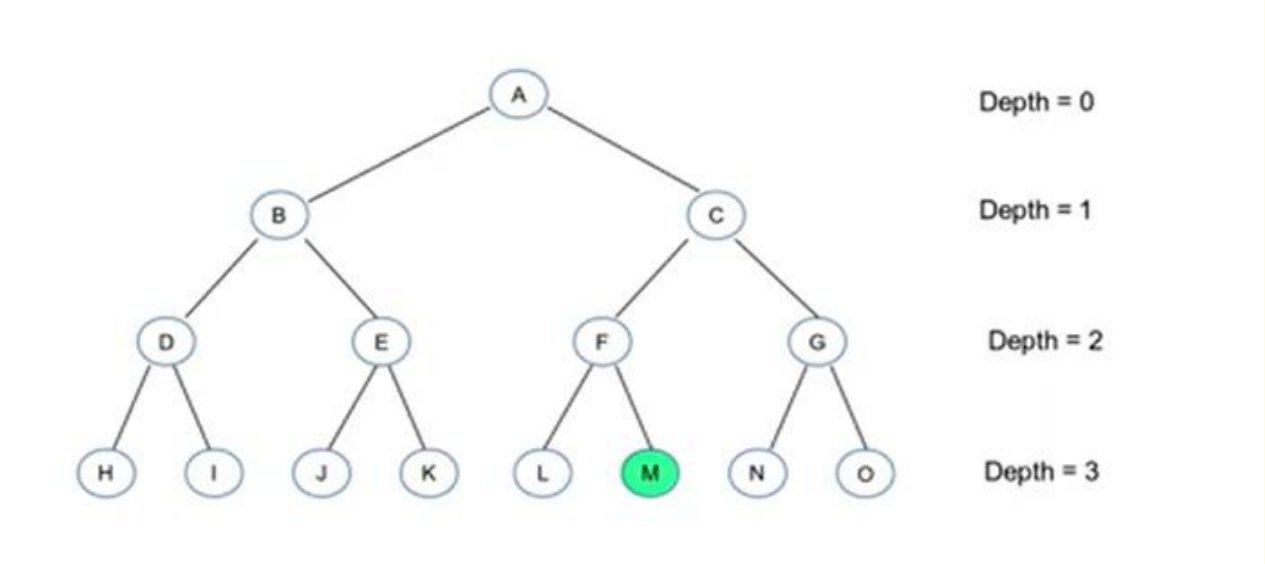
- **Key Components**

- Frontier (Stack): Stores nodes discovered but not yet expanded.
- Operates in LIFO order → Last-In-First-Out.
- Explored List (Visited Set):

  -Stores nodes already expanded so we don't revisit.

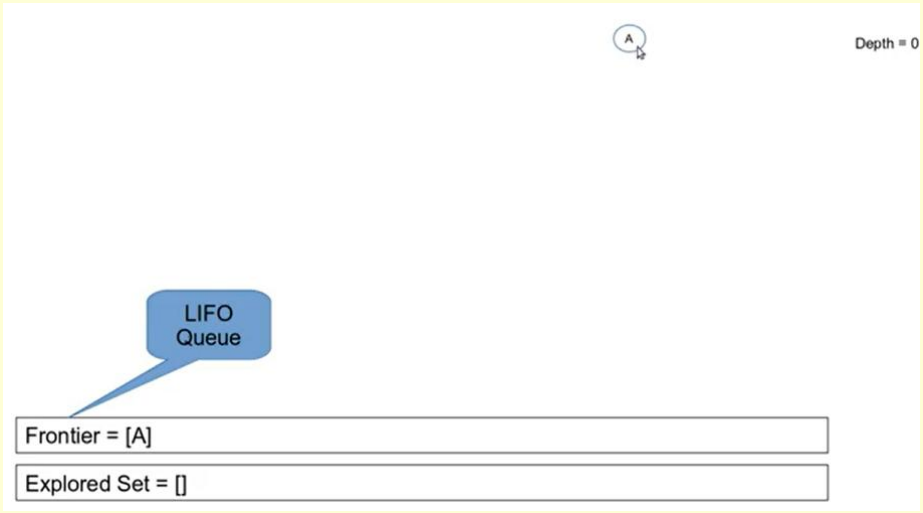
Depth-first search

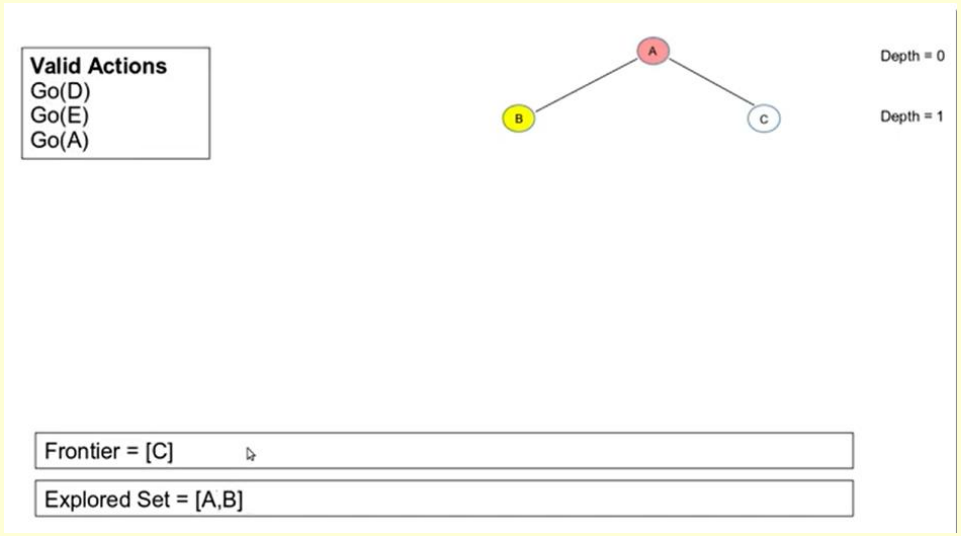
- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

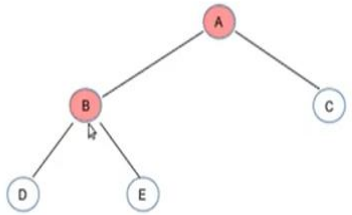
- Expand deepest unexpanded node
- Implementation:
  - *fringe* = LIFO queue, i.e., put successors at front





Depth-first search

Valid Actions
Go(D)
Go(E)
Go(A)

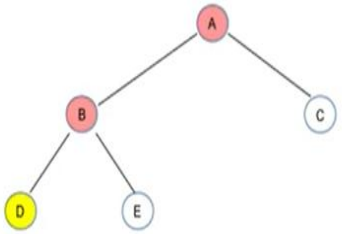


Depth = 0
Depth = 1
Depth = 2

Frontier = [D,E,C]

Explored Set = [A,B]

Valid Actions
Go(H)
Go(I)
Go(B)



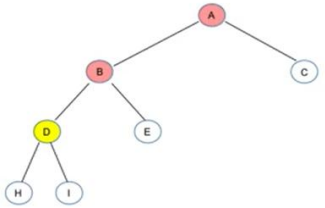
Depth = 0
Depth = 1
Depth = 2

Frontier = [E,C]

Explored Set = [A,B,D]

Depth-first search

Valid Actions
Go(H)
Go(I)
Go(B)

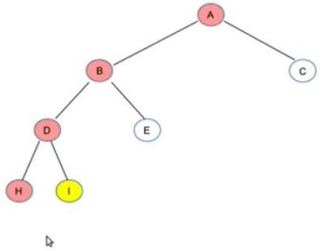


Depth = 0
Depth = 1
Depth = 2
Depth = 3

Frontier = [H,I,E,C]

Explored Set = [A,B,D]

Valid Actions
Go(D)

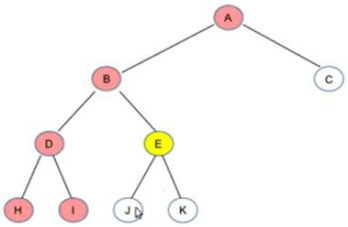


Depth = 0
Depth = 1
Depth = 2
Depth = 3

Frontier = [E,C]

Explored Set = [A,B,D,H,I]

Valid Actions
Go(J)
Go(K)
Go(B)

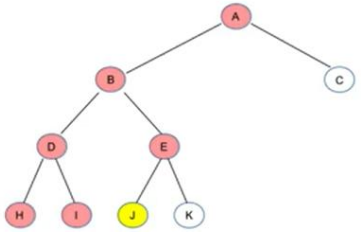


Depth = 0
Depth = 1
Depth = 2
Depth = 3

Frontier = [J,K,C]

Explored Set = [A,B,D,H,I,E]

Valid Actions
Go(E)



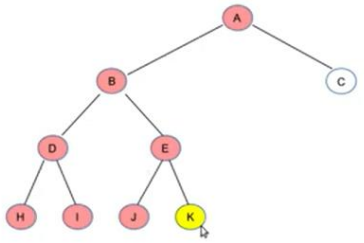
Depth = 0
Depth = 1
Depth = 2
Depth = 3

Frontier = [K,C]

Explored Set = [A,B,D,H,I,E,J]

Depth-first search

Valid Actions
Go(E)

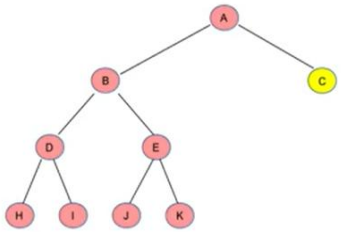


Depth = 0
Depth = 1
Depth = 2
Depth = 3

Frontier = [C]

Explored Set = [A,B,D,H,I,E,J,K]

Valid Actions
Go(F)
Go(G)
Go(A)

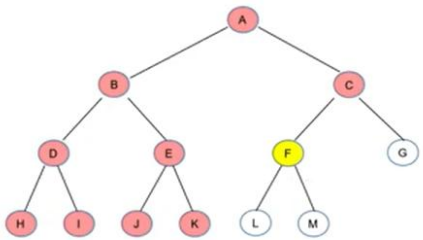


Depth = 0
Depth = 1
Depth = 2
Depth = 3

Frontier = []

Explored Set = [A,B,D,H,I,E,J,K,C]

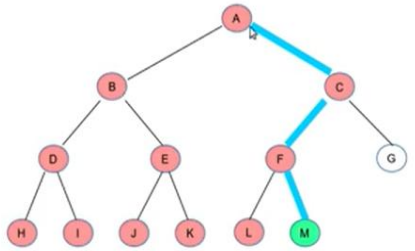
Valid Actions
Go(L)
Go(M)
Go(C)



Depth = 0
Depth = 1
Depth = 2
Depth = 3

Frontier = [L,M,G]

Explored Set = [A,B,D,H,I,E,J,K,C,F]



Depth = 0
Depth = 1
Depth = 2
Depth = 3

Reached a goal state!!

Return the solution: A –> C –> F –> M

Frontier = [G]

Explored Set = [A,B,D,H,I,E,J,K,C,F,L,M]

Depth-first search (Analysis)

- Not complete
  - because a path may be infinite or looping
  - then the path will never fail and go back try another option
- Not optimal
  - it doesn't guarantee the best solution because DFS chooses the path based on **depth**, not on **path cost**.
- It overcomes
  - the time and space complexities

Depth-first search Pros & Cons

- Advantages of Depth First Search (DFS)

1.**Less Memory Usage:** DFS stores only the nodes along the path from the root node to the current node,requiring less memory.

2.**Faster to Reach Goal**: It often takes less time to find a goal compared to Breadth First Search (BFS),especially when the solution is deep in the tree.

Depth-first search Pros & Cons

Disadvantages of Depth First Search (DFS)

1.**Recurring States:** Sometimes, many states repeat. In such cases, there's no guarantee of finding the solution.

2.Infinite Loops: DFS may get stuck in an infinite loop when it keeps going deeper. This can be avoided by setting an appropriate cut-off depth, but:

- Too small a cut-off may make the algorithm fail.
- Too large a cut-off increases execution time.

3.Complexity: The algorithm's complexity depends on the number of paths it needs to explore.

4.Duplicate Nodes: DFS cannot check for duplicate nodes, potentially leading to inefficiency in the search.

Properties of depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops
  → complete in finite spaces

- Time? $O(b^m)$: terrible if m is much larger than d
  - but if solutions are dense, may be much faster than breadth-first

- Space? $O(bm)$, i.e., linear space!

- Optimal? No

3. Uniform cost Search

Find the cheapest path from S (start) to G (goal) using Uniform Cost Search (UCS).

- UCS expands nodes based on **cumulative cost** from the start node.
- Always selects the node with the lowest total cost to explore next.

Algorithm (Graph Search version)

- Initialize a priority queue (min-heap) with the start node, cost = 0
- While queue is not empty:

  Remove the node with the lowest path cost (g(n)) (based on **Best First Search)**

  If this node is the goal, return the path.

  Otherwise, expand it → add successors with updated costs into the priority queue.
- Keep track of visited nodes (with best cost so far) to avoid re-expansion.

UCS Algorithm

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure

  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Figure 3.14 Uniform-cost search on a graph. The algorithm is identical to the general graph search algorithm in Figure 3.7, except for the use of a priority queue and the addition of an extra check in case a shorter path to a frontier state is discovered. The data structure for *frontier* needs to support efficient membership testing, so it should combine the capabilities of a priority queue and a hash table.

UCS

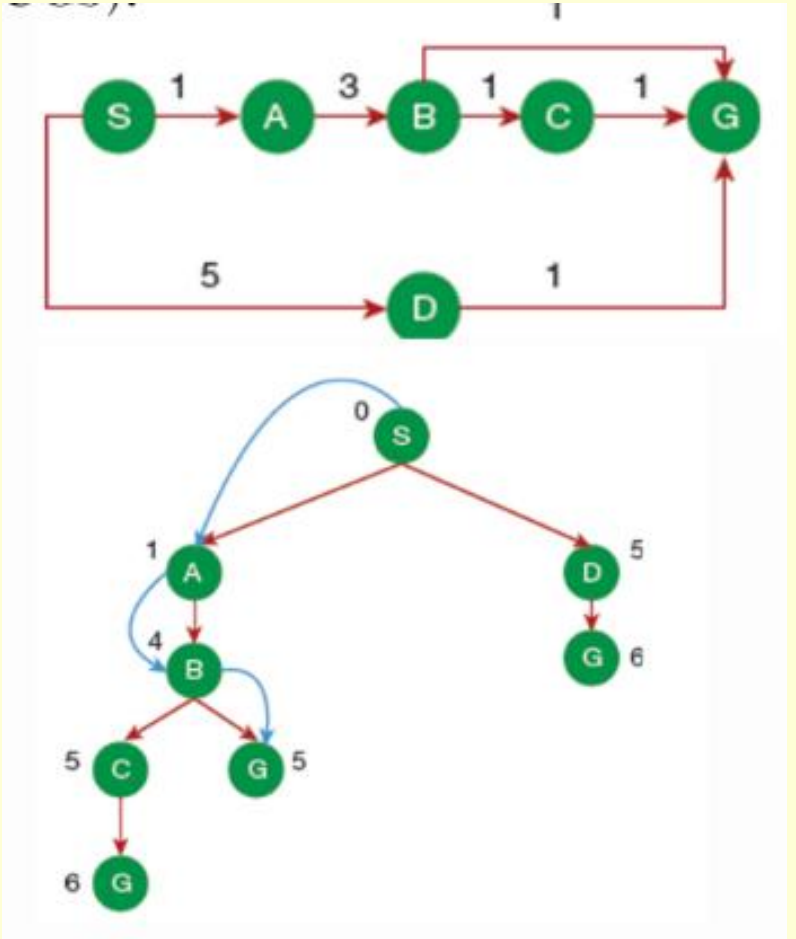
- Key points:

Selects Least Cumulative sum from root to goal node

Uses Priority Q as fringe list

Uniform cost Search

- **Step-by-Step Traversal:**
  - -Start at S → check child nodes A and other options.
  - -Choose A because it has the lowest cumulative cost.
  - -From A, explore B → cost is still lowest.
  - -From B, reach G → goal reached with least total cost.
- **Path Found:** S → A → B → G
- **Total Cost:** 5
- **Key Idea:** UCS always finds the optimal path based on actual costs, not depth or heuristic.



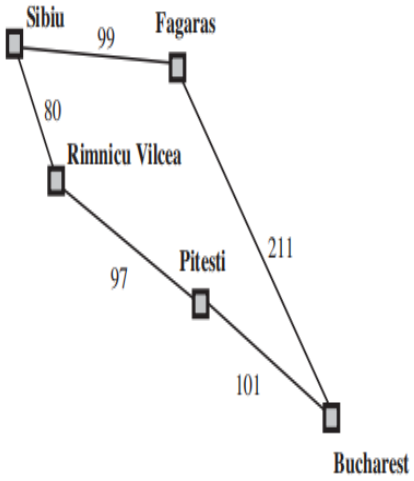
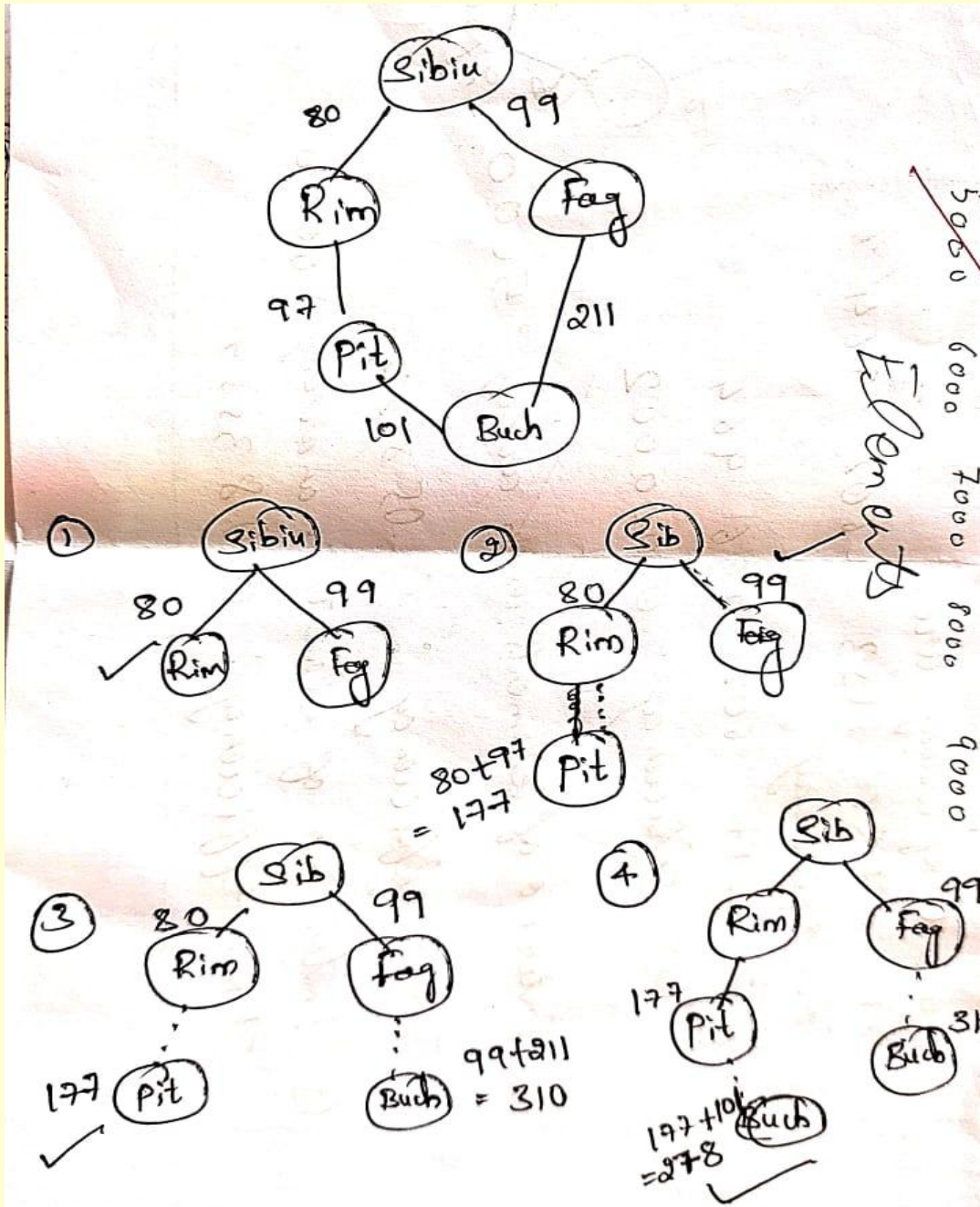


Figure 3.15 Part of the Romania state space, selected to illustrate uniform-cost search.



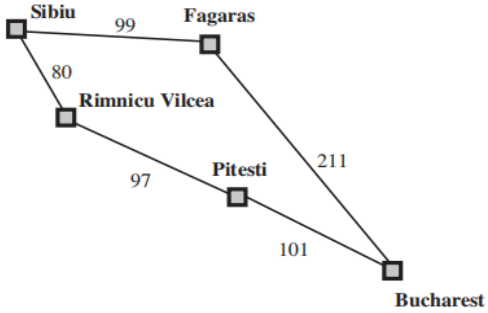
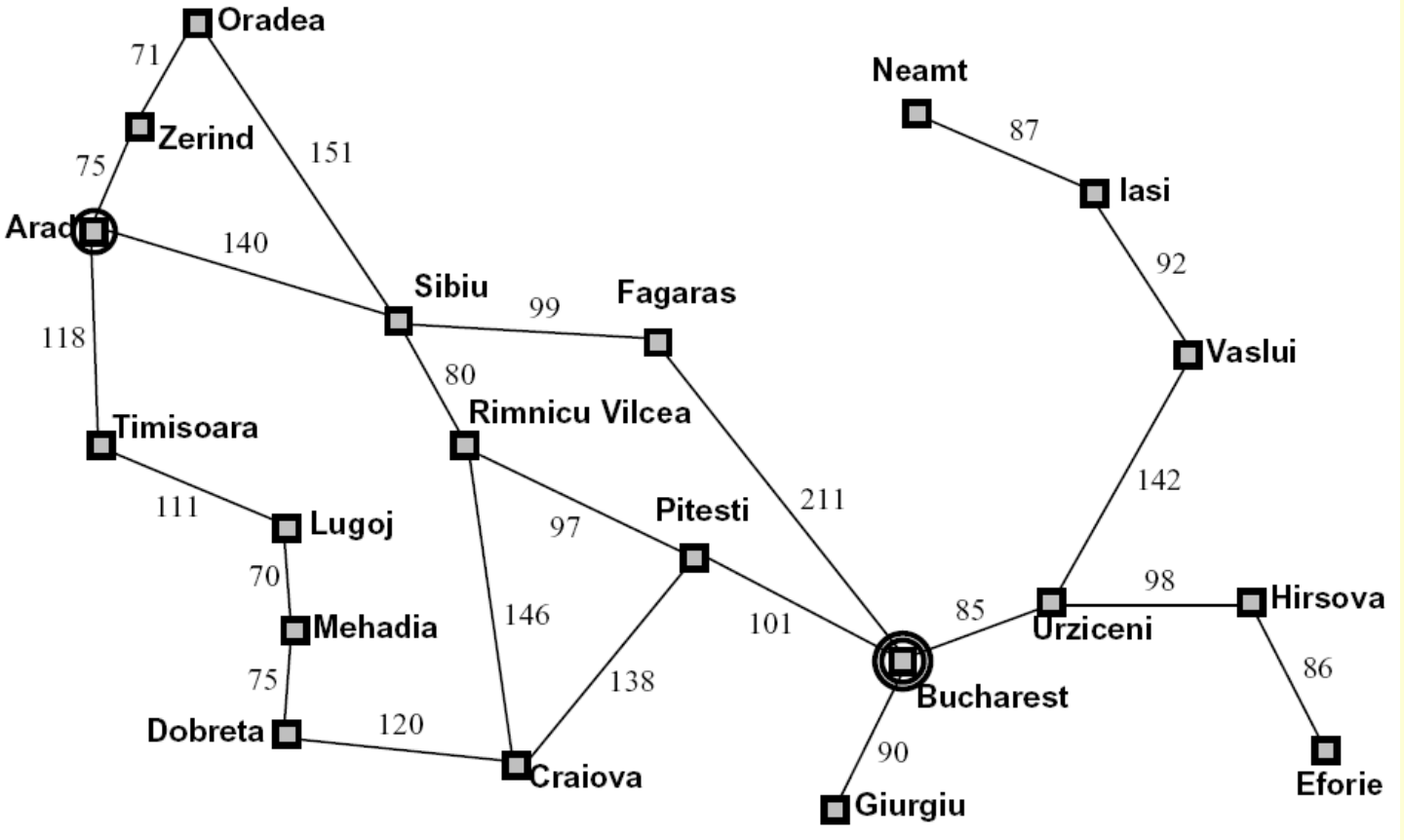


Figure 3.15 Part of the Romania state space, selected to illustrate uniform-cost search.

Frontier[]	Action	Explored[]	Cost g(x)
Siblu	Go[Rim],Go[Faga]		
		Siblu	g(Rim)=80 g(Faga)=99
Rim, Faga	Go[Pit], Go[sib]		
		Siblu, Rim	g(Faga)=99 g[Pit]=80+97=177
Faga,Pit	Go[Buch], Go[Sib]	siblu,Rim, Faga	g(pit)=177 g(buch)=310
Pit, Buch	Go[Rim], Go[Buch]	Siblu,Rim,Faga, Pit	g(buch)=80+97+101 =278
Buch	Goal with small cost stop		

Example


Oradea
71
Neamt
87
Zerind
151
75
Iasi
Arad
140
92
Sibiu
99
Fagaras
118
Vaslui
80
Rimnicu Vilcea
Timisoara
142
111
Pitesti
211
Lugoj
97
98
70
Hirsova
Mehadia
146
101
85
Urziceni
75
86
138
Bucharest
Dobreta
120
90
Craiova
Eforie
Giurgiu

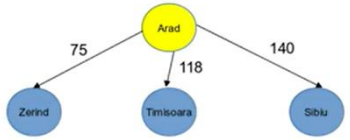
Valid Actions
Go(Zerind)
Go(Timisoara)
Go(Sibiu)



Frontier = []

Explored Set = [Arad]

Valid Actions
Go(Zerind)
Go(Timisoara)
Go(Sibiu)



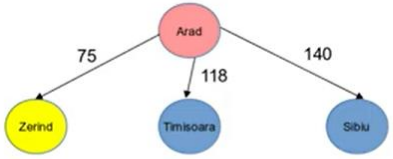
g(Zerind) = 75
g(Timisoara) = 118
g(Sibiu) = 140

Zerind < Timisoara < Sibiu

Frontier = [Zerind, Timisoara, Sibiu]

Explored Set = [Arad]

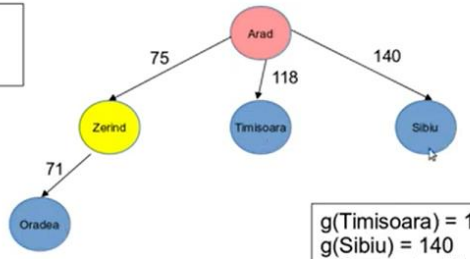
Valid Actions
Go(Arad)
Go(Oradea)



Frontier = [Timisoara, Sibiu]

Explored Set = [Arad, Zerind]

Valid Actions
Go(Arad)
Go(Oradea)



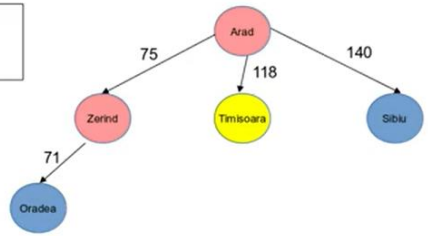
g(Timisoara) = 118
g(Sibiu) = 140
g(Oradea) = 75 + 71 = 146

Timisoara < Sibiu < Oradea

Frontier = [Timisoara, Sibiu, Oradea]

Explored Set = [Arad, Zerind]

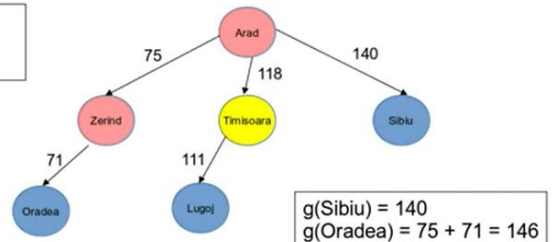
Valid Actions
Go(Arad)
Go(Lugoj)



Frontier = [Sibiu, Oradea]

Explored Set = [Arad, Zerind, Timisoara]

Valid Actions
Go(Arad)
Go(Lugoj)



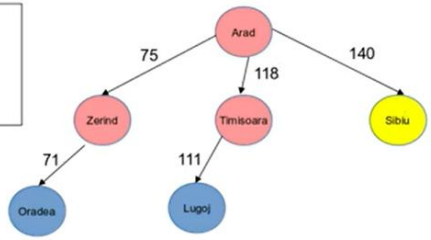
g(Sibiu) = 140
g(Oradea) = 75 + 71 = 146
g(Lugoj) = 118 + 111 = 229

Sibiu < Oradea <Lugoj

Frontier = [Sibiu, Oradea, Lugoj]

Explored Set = [Arad, Zerind, Timisoara]

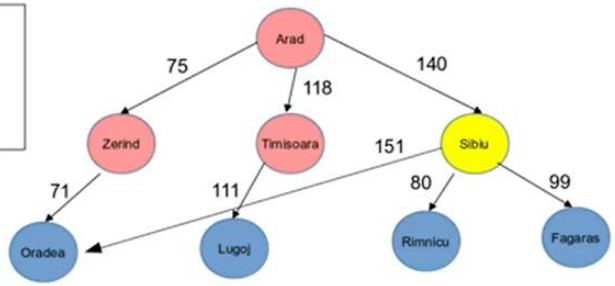
Valid Actions
Go(Arad)
Go(Oradea)
Go(Rimnicu)
Go(Fagaras)



Frontier = [Oradea, Lugoj]

Explored Set = [Arad, Zerind, Timisoara, Sibiu]

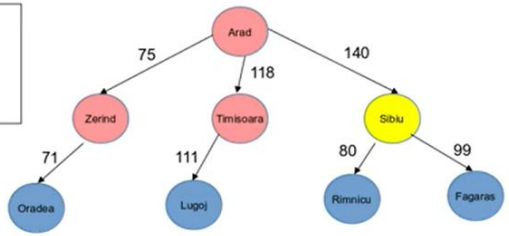
Valid Actions
Go(Arad)
Go(Oradea)
Go(Rimnicu)
Go(Fagaras)



Frontier = [Oradea, Lugoj]

Explored Set = [Arad, Zerind, Timisoara, Sibiu]

Valid Actions
Go(Arad)
Go(Oradea)
Go(Rimnicu)
Go(Fagaras)

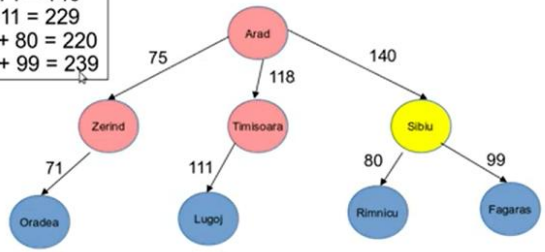


Arad -> Zerind -> Oradea = 75 + 71 = 146
Arad -> Sibiu -> Oradea = 140 + 151 = 291

Frontier = [Oradea, Lugoj]

Explored Set = [Arad, Zerind, Timisoara, Sibiu]

g(Oradea) = 75 + 71 = 146
g(Lugoj) = 118 + 111 = 229
g(Rimnicu) = 140 + 80 = 220
g(Fagaras) = 140 + 99 = 239

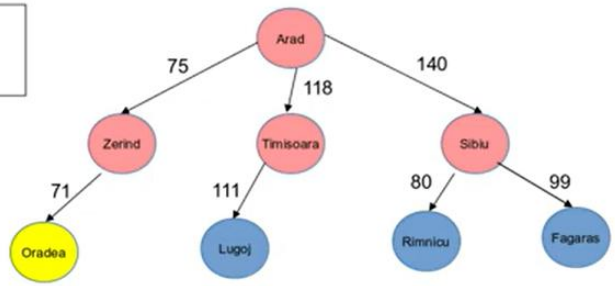


Oradea < Rimnicu < Lugoj < Fagaras

Frontier = [Oradea, Rimnicu, Lugoj, Fagaras]

Explored Set = [Arad, Zerind, Timisoara, Sibiu]

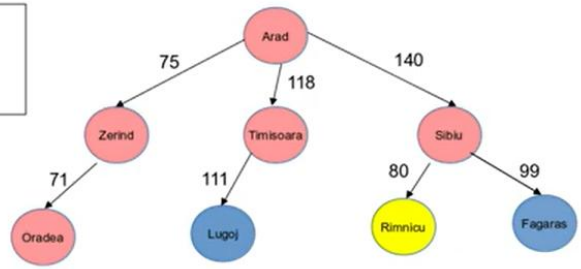
Valid Actions
Go(Zerind)
Go(Sibiu)



Frontier = [Rimnicu, Lugoj, Fagaras]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea]

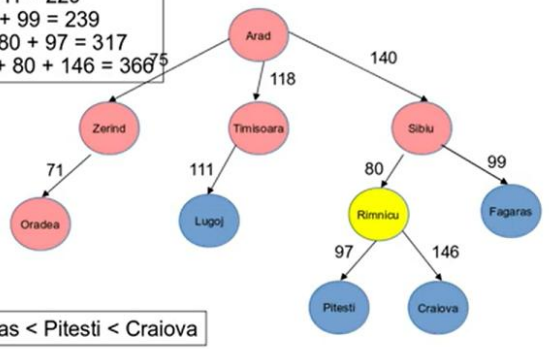
Valid Actions
Go(Sibiu)
Go(Pitesti)
Go(Craiova)



Frontier = [Logoj, Fagaras]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu]

g(Lugoj) = 118 + 111 = 229
g(Fagaras) = 140 + 99 = 239
g(Pitesti) = 140 + 80 + 97 = 317
g(Craiova) = 140 + 80 + 146 = 366

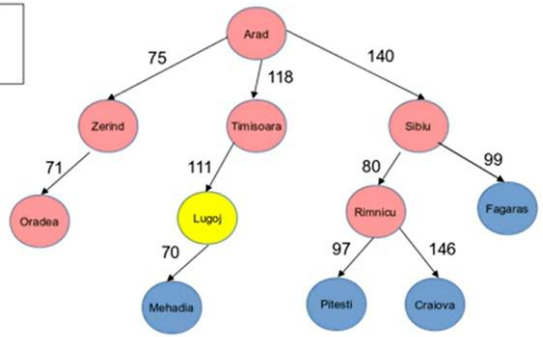


Lugoj < Fagaras < Pitesti < Craiova

Frontier = [Logoj, Fagaras, Pitesti, Craiova]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu]

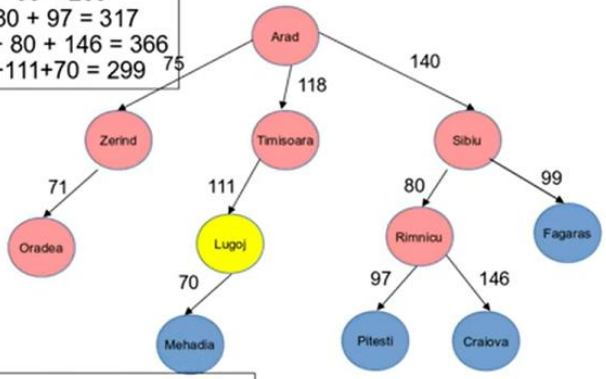
Valid Actions
Go(Timisoara)
Go(Mehadia)



Frontier = [Logoj, Fagaras, Pitesti, Craiova]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu, Lugoj]

g(Fagaras) = 140 + 99 = 239
g(Pitesti) = 140 + 80 + 97 = 317
g(Craiova) = 140 + 80 + 146 = 366
g(Mehadia) = 118+111+70 = 299

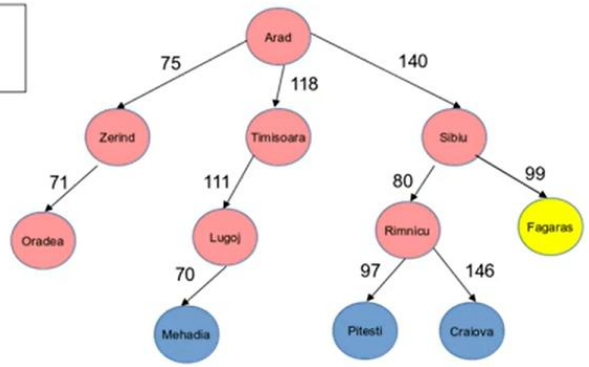


Fagaras < Mehadia < Pitesti < Craiova

Frontier = [Fagaras, Mehadia, Pitesti, Craiova]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu, Lugoj]

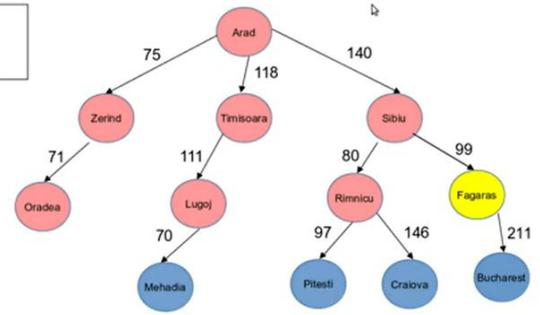
Valid Actions
Go(Sibiu)
Go(Bucharest)



Frontier = [Mehadia, Pitesti, Craiova]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu, Lugoj, Fagaras]

Valid Actions
Go(Sibiu)
Go(Bucharest)

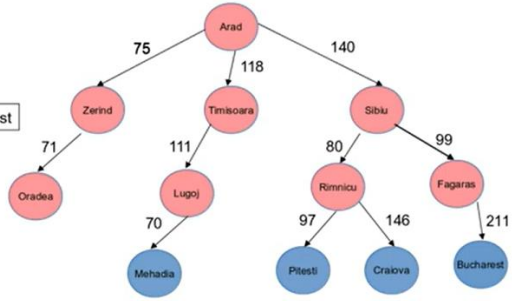


Frontier = [Mehadia, Pitesti, Craiova]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu, Lugoj, Fagaras]

g(Pitesti) = 140 + 80 + 97 = 317
g(Craiova) = 140 + 80 + 146 = 366
g(Mehadia) = 118+111+70 = 299
g(Bucharest) = 140 + 99 + 211 = 450

Mehadia < Pitesti < Craiova < Bucharest

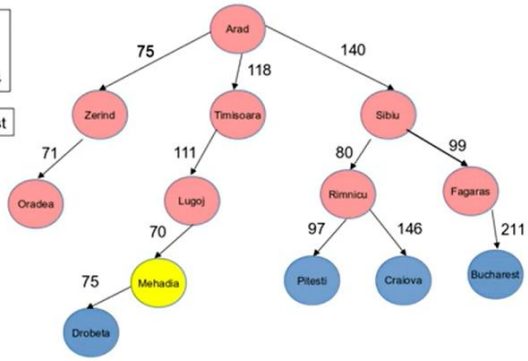


Frontier = [Mehadia, Pitesti, Craiova, Bucharest]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu, Lugoj, Fagaras]

g(Pitesti) = 140 + 80 + 97 = 317
g(Craiova) = 140 + 80 + 146 = 366
g(Bucharest) = 140 + 99 + 211 = 450
g(Drobeta) = 118 + 111 + 70 + 75 = 374

Pitesti < Craiova < Drobeta < Bucharest



Frontier = [Pitesti, Craiova, Drobeta, Bucharest]

Explored Set = [Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu, Lugoj, Fagaras, Mehadia]

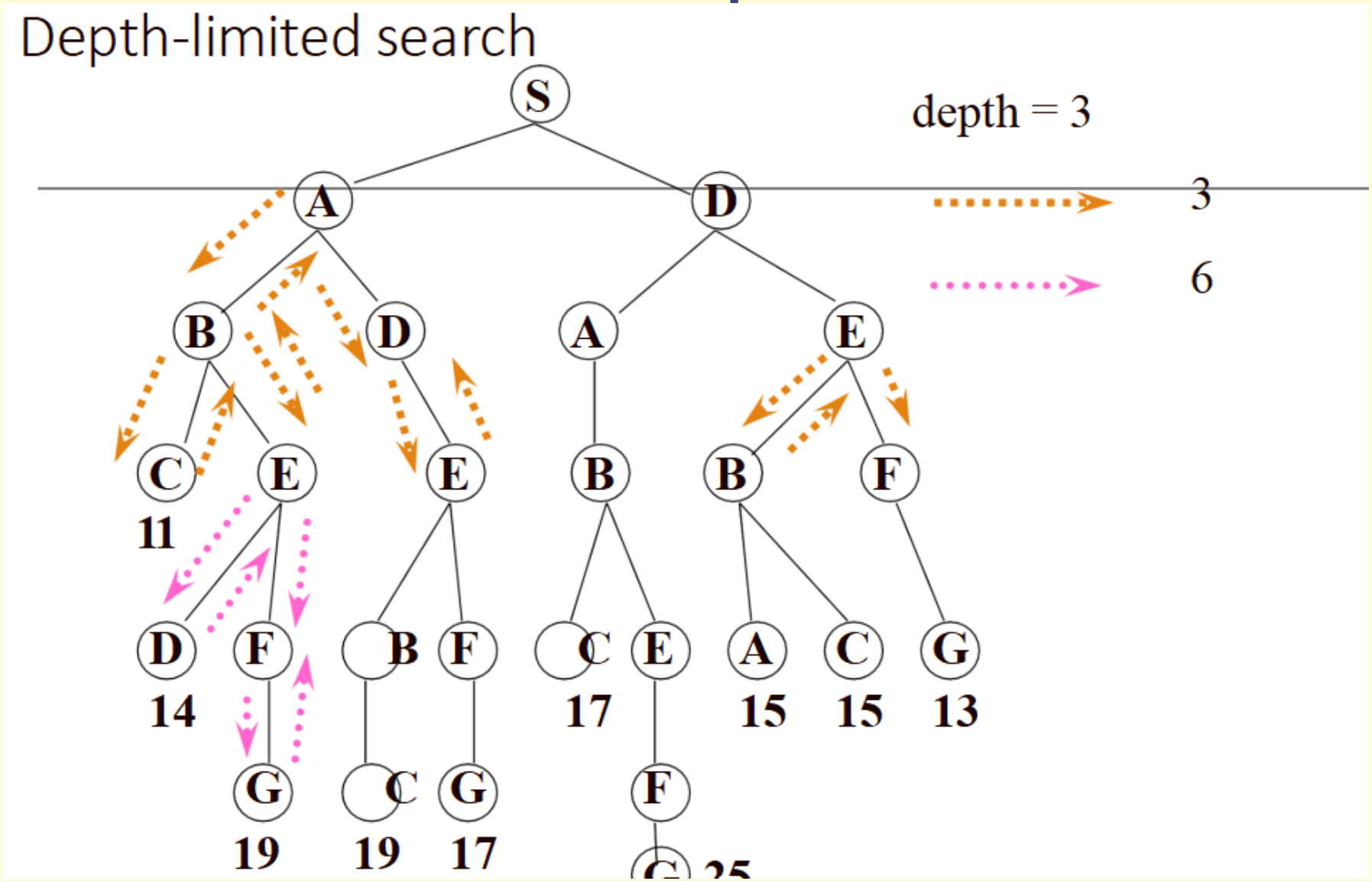
UCS performance

- **Complete:** guaranteed provided the cost of every step exceeds some small positive constant ε
- **Optimal:** optimal path to that node has been found. because step costs are nonnegative, paths never get shorter as nodes are added. uniform-cost search expands nodes in order of their optimal path cost.
- **Time Complexity**: # of nodes with $g \leq$ cost of optimal solution, $O(b^{ceiling(C^*/\varepsilon)})$
- where C^* is the cost of the optimal solution
- **Space Complexity**: # of nodes with $g \leq$ cost of optimal solution, $O(b^{ceiling(C^*/\varepsilon)})$

4.Depth-limited search

- Depth limited search is a Depth-first search with a predetermined depth limit l. *N*odes at depth l are treated as if they have no successors. The depth limit solves the infinite-path problem.
- Useful when we know an approximate depth of the solution.
- How it Works?
  - Start from the root node (initial state).
  - Expand nodes in a depth-first manner.
  - If the depth of the node ≤ depth limit l, expand further.
  - If the depth exceeds l, stop exploring that branch.
  - If a goal is found before reaching the limit, return the solution.

Ex. at depth = 3


Depth-limited search
S
depth = 3
A
D
3
6
B
D
A
E
C
E
E
B
B
F
11
D
F
B
F
C
E
A
C
G
14
17
15
15
13
G
C
G
F
19
19
17
25

Depth Limited Search Algorithm

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
      cutoff_occurred? ← false
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          result ← RECURSIVE-DLS(child, problem, limit − 1)
          if result = cutoff then cutoff_occurred? ← true
          else if result ≠ failure then return result
      if cutoff_occurred? then return cutoff else return failure
```

Figure 3.17 A recursive implementation of depth-limited tree search.

Performance of DLS

- DLS introduces an additional source of incompleteness if we choose l<d, that is, the shallowest goal is beyond the depth limit.
- Ex: suppose we set a depth limit = 3:
  - If the solution lies at depth 2 → DLS will find it.
  - If the solution lies at depth 5 → DLS will fail.
- Depth-limited search will also be non optimal if we choose l >d. Its time complexity is $O(b^l)$ and its space complexity is $O(b^l)$.
- Depth-first search can be viewed as a special case of depth-limited search with $l=\infty$.

5.Iterative deepening depth- first search

Iterative deepening search is often used in combination with depth-first tree search, that finds the best depth limit.

It does this by gradually increasing the limit—first 0, then 1, then 2, and so on until a goal is found.

This will occur when the depth limit reaches d, the depth of the shallowest goal node

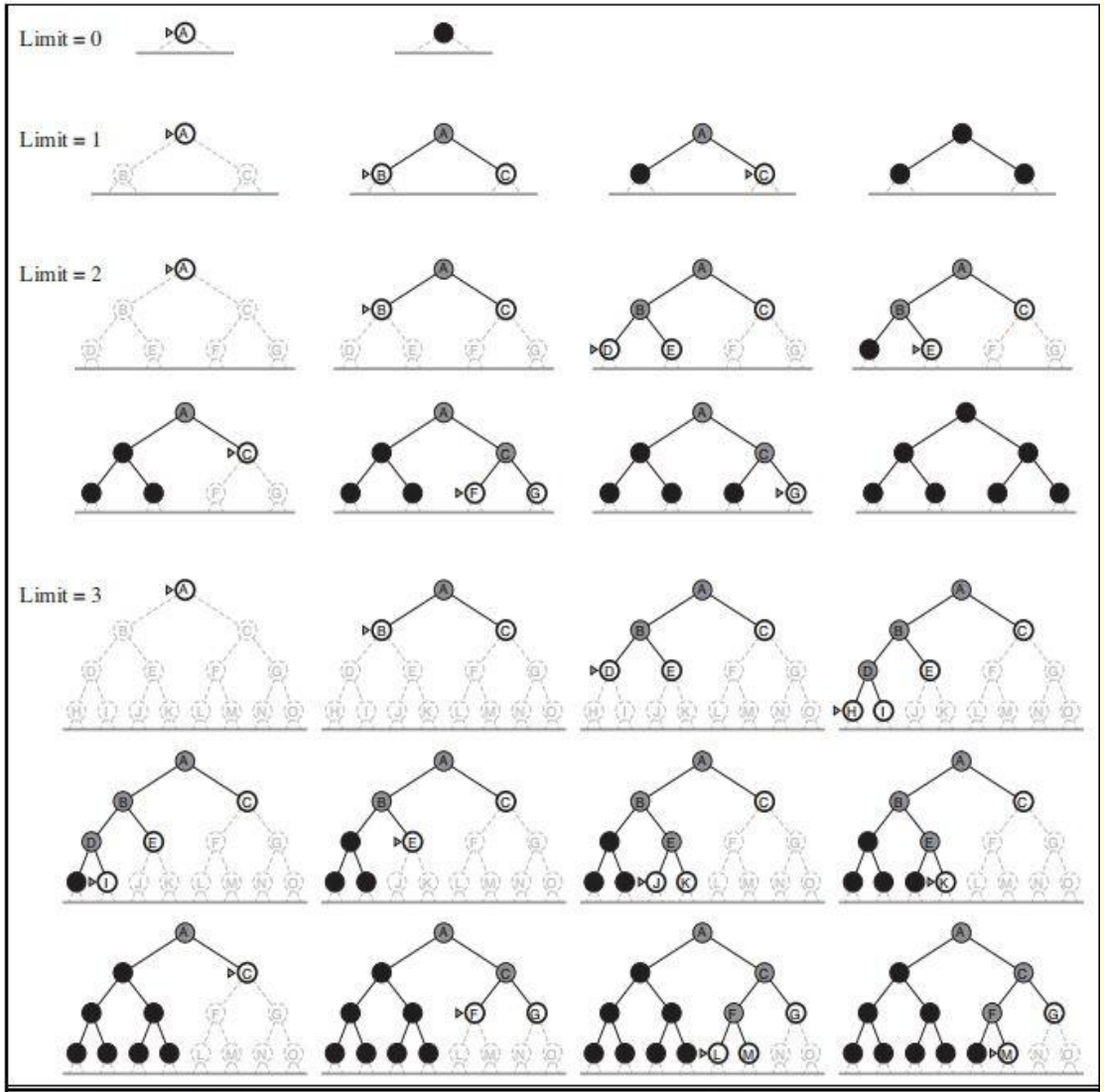


Figure 3.19 Four iterations of iterative deepening search on a binary tree.

Contd.,

It combines the space efficiency of Depth-First Search (DFS) with the completeness of Breadth-First Search (BFS).

How it Works

- Start with depth limit = 0.
- Run Depth-Limited Search (DLS).
- If goal not found → increase limit by 1.
- Repeat until solution is found.

The total number of nodes generated in the worst case is:

- $N(IDS)=(d)b + (d-1)b^2 + \cdots + (1)b^d$,

which gives a time complexity of $O(b^d)$ asymptotically the same as breadth-first search.

Iterative deepening search is analogous to breadth-first search in that it explores a complete layer of new nodes at each iteration before going on to the next layer

iterative deepening is the preferred uninformed search method when the search space is large and the depth of the solution is not known

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
    for depth = 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH(problem, depth)
        if result ≠ cutoff then return result
```

Figure 3.18 The iterative deepening search algorithm, which repeatedly applies depth-limited search with increasing limits. It terminates when a solution is found or if the depth-limited search returns *failure*, meaning that no solution exists.

Permormance of IDDFS

Completeness : YES

Always finds a solution if one exists, because the depth keeps increasing.

Optimality : YES

Finds the shallowest (minimum depth) solution, just like BFS.

Time Complexity:

$O(bd)$O(bd)

where b = branching factor, d = depth of shallowest solution.

Space Complexity:

$O(b \cdot d)$O(b·d) (linear like DFS).